

Fault Detection and Diagnosis in Building Heating and Cooling Patterns

Master of Data Analytics

CS7P01 MSc Project-Spring 2023-24

Author: Deshpande Padmaja Rohan

Student ID No: 22044134

Supervisor: Dr. Maitreyee Dey



London Metropolitan University Faculty of Computing
School of Computing and Digital Media

Abstract:

Building managers deal with the problem of fan coil unit (FCU) failure in HVAC (heating, ventilation, and air conditioning) systems in buildings. Because of these flaws, buildings lose 15–30% of the energy that the FCUs use. This thesis looks at fault detection and diagnostics to pinpoint the precise fault in the FCU unit to prevent energy waste. After pre-processing and data transformation when the data was finally ready, machine learning algorithms that attempted to find faults included K-nearest neighbour (KNN), support vector machine (SVM), (MLP)multilayer perceptron, and deep multilayer perceptron (DEEP MLP). This model used several splits, such as 70-30 % and 80-20 %, to verify the results of all of the models. But in order to prevent the model from over-fitting, the term "label noise" was included in the present case. As a result, the model operates both with and without label noise. Additionally, experiment with a different kernel and optimizer to get better results. All of these models were evaluated using the confusion matrix, F1 score, recall and precision. Among all these models, the MLP model was the most established since it consistently achieved high testing and training accuracies of 0.99 with and without noise 80-20% sample split. This model's robustness in recognising intricate patterns in the data while maintaining its capacity to generalise to new cases was demonstrated by its low training and testing losses. Moreover, the addition of the Adam optimizer and two hidden layers enhanced the MLP's precision in distinguishing between fault and non-fault instances. Consequently, the analysis indicates that the MLP model is the most effective choice for fault identification.

Key words: FCU unit, fault detection, machine learning etc.

Acknowledgement:

I would want to thank the University for assigning me a capable tutor to help me finish my project. I also want to express my sincere gratitude to my mentor, Dr. Maitreyee, for all of her help and insightful advice during this project. She is always willing to help, and her prompt responses inspire me to always provide my best effort on projects. She also assisted me in polishing my thesis-writing abilities.

Finally, I would like to give my family my sincere gratitude: my son Harshit and husband Rohan. Their love and positive energy have been a major source of contribution to this project. My backbones are both of them. Every successful woman has a supportive spouse by her side. My spouse's assistance has allowed me to finish my project.

Table of Contents

Chapter 1. Introduction	10
1.1 Introduction:	10
1.2 Statement of Problem:	10
1.3 Research Aim:	11
1.4 Proposed solution:	11
1.5 Outline of the study:	11
Chapter 2: Literature Review	12
2.2 Literature Review:	12
Chapter 3: Methodology	17
3.1 Data collection and Exploration:	17
3.2 FCUs unit:	18
3.3 Faults:	19
3.4 Types of units and faults in FCU unit:	19
3.5 Threshold:	20
3.6 Data Preprocessing:	21
3.6.1 Data cleaning and transformation:	21
3.6.2 Label encoder:	22
3.7 Noise:	22
3.8 Data availability and size:	23
3.9 Machine learning algorithm:	23
3.9.1 Support vector machine:	24
3.9.2 K-Nearest Neighbors (KNN):	25
3.9.3 Multilayer Perceptron (MLP):	26
3.9.4 Deep Multilayer Perceptron (Deep MLP):	27
3.10 Model Evaluation:	28
3.10.1 Confusion Matrix:	28
3.10.2 Precision:	28
3.10.3 Recall:	28
3.10.4 F1 Score:	29
Chapter 4 Analysis and Design	29
4.1 Data Description:	29
4.2 Exploratory analysis:	29
4.2.1 Histogram:	30
4.2.2 Count Plot:	30
4.2.3 Heatmap:	31

4.2.4 Count plot:	32
4.3 Visualization by Power Bi:	33
Chapter 5: Result and Analysis	34
5.1 Result of KNN:	34
5.1.1 KNN without noise with 70-30%:.....	34
5.1.2 KNN model with 80-20% split without Noise:	35
5.1.3 KNN model with 70-30% split with Noise:	36
5.1.4 KNN with 80-20 split with Noise:	37
5.1.5 KNN with 70-30% split without noise (N=5):.....	38
5.1.6 KNN with 80-20% split without noise (N=5):.....	40
5.1.7 KNN with 70-30% split with noise (N=5):	40
5.1.8 KNN with 80-20% split with noise (N=5):	41
5.2 Result of SVM:	42
5.2.1 SVM with 70-30% without noise:.....	42
5.2.2 SVM with 80-20% without noise:.....	43
5.2.3 SVM with 70-30 split with Noise:	44
5.2.4 SVM with 80-20 split with Noise:	45
5.3 Result of MLP:	46
5.3.1 MLP with 70-30 split without noise:.....	46
5.3.2 MLP with 80-20% split without noise:.....	47
5.3.3 MLP with 70-30% split with noise:	47
5.3.4 MLP with 80-20% split with noise:	48
5.3.5 MLP with 70-30% split without noise & SGD Optimizer:.....	49
5.3.6 MLP with 80-20% split without noise & SGD Optimizer:.....	49
5.3.7 MLP with 70-30% split with noise & SGD Optimizer:	50
5.3.8 MLP with 80-20% split with noise & SGD Optimizer:	50
5.4 Result of DEEP MLP:.....	50
5.4.1 DEEP MLP with 70-30% split without noise:	50
5.4.2 DEEP MLP with 80-20% split without noise:	51
5.4.3 DEEP MLP with 70-30% split with noise:.....	51
5.4.4 DEEP MLP with 80-20% split with noise:.....	52
5.5 Class Imbalance:	52
Chapter 6: Conclusion and future work	52
6.1 Conclusion:	52
6.2 Future Work:.....	53
References:	54

<i>Appendix:</i>	57
------------------------	----

List of Figures

Figure 1: Outline of the study.	12
Figure 2: Diagrammatical representation of data.	18
Figure 3: Typical FCU unit diagram [30].	19
Figure 4: Diagrammatical representation types of faults in FCU unit.....	20
Figure 5: Steps used for data cleaning and transformation.....	22
Figure 6: Diagrammatical representation of label noise	23
Figure 7: Working of SVM by diagram	25
Figure 8: diagrammatical representation of how KNN works.....	26
Figure 9: Diagrammatical representation of MLP model with two hidden layers.	27
Figure 10: Working of DEEP MLP model with five hidden layers.	28
Figure 11: Histogram of value column.	30
Figure 12: Count plot for label according to value.	31
Figure 13: overall correlation heatmap for all numeric columns.....	32
Figure 14: Count plot for attribute names.....	33
Figure 15: Power Bi desktop.	33

List of Tables

Table 1: General overview of confusion matrix.....	28
Table 2: Description of each attribute present in dataset.	29
Table 3: Accuracy table for KNN with 70-30% split without noise.....	34
Table 4: Training confusion matrix for KNN with 70-30% split without noise	35
Table 5: Testing confusion matrix for KNN with 70-30% split without noise	35
Table 6: Accuracy table for KNN 80-20 split without noise	35
Table 7: Training confusion matrix for KNN 80-20 split without noise.....	36
Table 8: Testing confusion matrix for KNN with 80-20 split without noise.	36
Table 9: Accuracy table for KNN with 70-30 split using noise.....	36
Table 10: Training confusion matrix for KNN with 70-30 split using noise.	37
Table 11: Testing confusion matrix for KNN with 70-30 split using noise.....	37
Table 12: Accuracy table for KNN with 80-20 split using noise.....	38
Table 13: Training confusion matrix for KNN with 80-20 split using noise.....	38
Table 14: Testing confusion matrix for KNN with 80-20 split using noise.....	38
Table 15: Accuracy table for KNN with 70-30 split without noise (N=5).....	39
Table 16: Training confusion matrix for KNN with 70-30 split without noise (N=5).....	39
Table 17: Testing confusion matrix for KNN with 70-30 split without noise (N=5).	39
Table 18: Accuracy table for KNN with 80-20 split without noise (N=5).....	40
Table 19: Training confusion matrix for KNN with 80-20 split without noise (N=5).....	40
Table 20: Testing confusion matrix for KNN with 80-20 split without noise (N=5).	40
Table 21: Accuracy matrix for KNN with 70-30 split using noise (N=5)	41
Table 22: Training confusion matrix for KNN with 70-30 split using noise (N=5).	41
Table 23: Testing confusion matrix for KNN with 70-30 split using noise (N=5).	41
Table 24: Accuracy table for KNN with 80-20 split using noise (N=5)	42
Table 25: Training confusion matrix for KNN with 80-20 split using noise (N=5).	42
Table 26: Testing confusion matrix for KNN with 80-20 split using noise (N=5).	42
Table 27: Accuracy table for SVM with 70-30 split without noise	43
Table 28: Training confusion matrix for 70-30 split without noise.	43
Table 29: Testing confusion matrix for 70-30 split without noise.....	43
Table 30: Accuracy table for SVM with 80-20 split without noise.	44
Table 31: Training confusion matrix for SVM with 80-20 split without noise.	44
Table 32: Testing confusion matrix for SVM with 80-20 split without noise.	44
Table 33: Accuracy table for SVM with 70-30 split with noise.	45
Table 34: Training confusion matrix for SVM with 70-30 split with noise.....	45
Table 35: Testing confusion matrix for SVM with 70-30 split with noise.	45
Table 36: Accuracy table for SVM with 80-20 split with noise.	46
Table 37: Training confusion matrix for SVM with 80-20 split with noise.....	46
Table 38: Testing confusion matrix for SVM with 80-20 split with noise.	46
Table 39: Accuracy table for MLP with 70-30 split without noise.....	47
Table 40: Accuracy table for MLP with 60-20 split without noise.....	47
Table 41: Accuracy table for MLP with 70-30 split with noise.....	48

Table 42: Accuracy table for MLP with 80-20 split with noise.....	48
Table 43: Accuracy table for MLP with 70-30 split without noise with SGD optimizer	49
Table 44: Accuracy table for MLP with 80-20 split without noise using SGD optimizer.....	49
Table 45: Accuracy table for MLP with 70-30 split with noise using SGD optimizer.....	50
Table 46: Accuracy table for MLP with 80-20 split with noise using SGD optimizer.....	50
Table 47: Accuracy table for DEEP MLP with 70-30 split without noise.....	51
Table 48: Accuracy table for DEEP MLP with 80-20 split without noise.....	51
Table 49: Accuracy table for DEEP MLP with 70-30 split with noise.....	51
Table 50: Accuracy table for DEEP MLP with 80-20 split with noise.....	52

Chapter 1. Introduction

1.1 Introduction:

HVAC systems account for a substantial portion of building energy use in business and institutional settings - more than 40% due to their high energy consumption. HVAC systems are frequently controlled by building automation systems to improve indoor comfort and building energy efficiency. A study conducted in the United Kingdom revealed that 25% to 50% of building energy is lost as a result of HVAC system problems [1]. According to studies, inefficient controls and broken equipment account for 15% to 30% of power loss in business premises. Building HVAC systems can have long-term, healthy operation guaranteed by early fault detection and diagnosis. Buildings' energy efficiency can be successfully increased, and energy savings of between 10% and 30% can be realised [2].

HVAC systems have seen the development of numerous techniques for malfunction diagnosis and detection, addressing different types of sensor and facility failures, based on Annex 25 and Annex 34 [3]. The International Energy Agency (IEA) assembled a variety of fault detection and diagnosis (FDD) methodologies and applications to further detect and prevent failures, while HVAC systems were intended to be monitored and controlled by Building Energy Management Systems (BEMSs) [4].

The FCU is an independent air handling unit with the required controls, a fan, and a heating or cooling coil. Air is drawn from the surrounding area, heated or cooled by the coil, and then distributed back into the room. This is how it works. Through the use of a valve system, the hot or cold-water flow through the coil is adjusted to achieve temperature management. FCUs are monitored for a number of characteristics in the context of FDD, including air flow rate, temperature variations across the coil, fan speed, and valve performance. Any departure from the anticipated performance parameters may point to a systemic flaw or inefficiency, which the FDD system may quickly detect and diagnose.

The cost-effectiveness and energy conservation of building operations, in addition to occupant comfort, depend heavily on the efficient operation of FCUs. Consequently, proactive maintenance, early failure detection, and accurate diagnosis are made possible by the integration of FDD systems with FCUs, which enhances system performance, lowers downtime, and saves energy.

The data used in this study is for a single building with many floors and a number of fan coil units (FCU) on each floor. The building has 731 FCUs installed, and each day, data is gathered from them.

1.2 Statement of Problem:

Optimizing interior climate control, energy efficiency, and occupant comfort in buildings is significantly hindered by the operation and maintenance of Fan coil Units (FCUs). Even though FCUs are widely used in HVAC systems, they frequently encounter a number of problems that affect their reliability and effectiveness.

High energy consumption efficiency is one of the main issues with FCUs. A significant number of FCUs run constantly at full capacity, wasting energy and raising utility bills. FCU calibration errors can also contribute to decreased efficiency and increased equipment damage, which can raise maintenance costs and shorten the equipment's lifespan.

The absence of useful monitoring and diagnostic tools for FCUs presents another difficulty. The manual inspection and reactive maintenance methods used by many current systems might cause delays in problem identification and resolution. Building operators find it difficult to maximize FCU performance and avoid expensive breakdowns in the absence of live observation and early fault-finding capabilities.

In conclusion, the maintenance and functioning of FCUs in buildings present a complex issue that includes issues with energy inefficiency, equipment dependability, fault identification, and integration. Improving interior climate management, cutting energy use, saving maintenance costs, and raising occupant happiness and comfort all depend on addressing these problems.

1.3 Research Aim:

The comprehensive objective of this project is to improve energy economy, system reliability, and customer satisfaction by creating and implementing an efficient building heating failure detection and diagnostic (FDD) system and cooling systems. It is feasible to identify and remove the defects and their predictors as quickly as possible by using machine learning classification techniques. Despite having a better fault detection accuracy, this method is simple. As buildings become increasingly intelligent and produce large amounts of data as their primary input, this method is being employed more and more to solve find defects in FCUs.

1.4 Proposed Solution:

The study proposed fault detection in the fan and coil unit. For the purpose of defect identification, the used method is machine learning algorithm, especially classification algorithm. Given real time data to, the machine convert the data into ten feature vectors for analysis purposes and gets better result. Using a threshold value, the machine distributes data into faulty and non-faulty categories which further helps in analysis. Moreover, machines label them as faulty as 1 and non-faulty as 0. After that, we used multi-layer preceptors (MLP), deep multi-layer preceptors (DEEP MLP), KNN, or the k-nearest neighbour, and SVM support vector machines. This is because classification, which is important for resolving real-world issues like fault detection in FCU units, involves assigning a pattern to a particular class that is already known a priori. Classification is used to identify errors using this data, which supports the use of classification in problem solving. In terms of expenses, effort, and complexity, this study offers a less expensive solution.

1.5 Outline of the study:

The study is structured as follows shown in figure:

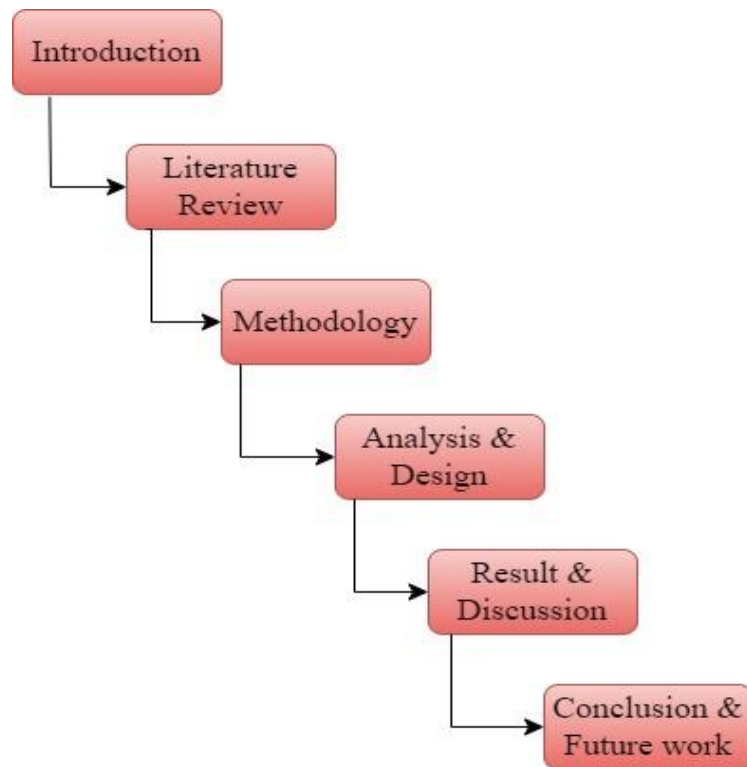


Figure 1:Outline of the study.

The literature review is covered in Chapter 2, and the study methodology, types of faults in FCUs, pre-processing, and label noise used for analysis are covered in Chapter 3. Data analytics and data attribute design are discussed in Chapter 4. Additionally, Chapter 5 covers the results and evaluation of machine learning models, and Chapter 6 concludes with recommendations for further research and work.

Chapter 2: Literature Review

2.2 Literature Review:

In one work, regression-based supervised machine learning was utilised by Yang Zhao, Chaobo Zhang, Yiwen Zhang, Zihao Wang, and Jun yang Li to forecast the building energy demand. Again, utilising an artificial neural network (ANN) technique, they created energy load prediction. The three main levels of a neural network (ANN) are the input coat, one or more hidden layers, and the output coat. Artificial neurons are present in every layer and are connected to those in layers above and below. Finally, it can be said that the supervised data mining-based approaches performed well in predicting building energy loads and have the benefit of not requiring domain expertise or physical models. In addition, their forecast accuracy is extremely high [5].

An additional investigation into the diagnosis and fault-finding of HVAC systems was carried out by Jianli Chen, Liang Zhang, Yanfei Li, Yifu Shi, Xinghua Gao, and Yuqing Hu. They used a variety of analytical techniques, including knowledge-based approaches. For FDD, this analysis-based method makes use of technical expertise regarding defect symptoms and system settings. An example of this is the Bayesian Network method. The associations between variables are represented by nodes and edges in the probabilistic graphical model known as a Bayesian network. Additionally, SVM is used in supervised machine learning algorithms to

create hyperplanes that divide space into distinct regions that indicate fault and fault-free operating status. SVM is used to produce support vectors during algorithm training. Only supervised approaches are assessed because of a shortage of data. Once more, they identify sensor faults using unsupervised learning techniques like PCA or clustering [6].

Hua Wang, Enbo Yan, et al. conducted a third study. Jian Bi. They categorised AI-based HVAC FDD techniques into traditional ML-based and DL-based groups in order to analyse the use of AI in HVAC diagnosis and fault detection. Within the field of machine learning, the k points that are nearest to the sample are chosen using the KNN model. The KNN method is simple to use and doesn't involve any pre-processing of the data or parameter adjustments. The degree of abnormality in the sample is then ascertained using the PCA method. In the FDD technique based on SVM, the data is then changed into an environment with more dimensions by the use of a kernel function. Subsequently, they utilised methods that involved artificial neural networks. Fault classes are typically allocated to the final layer while the input layer is designated to receive the ANN feature variables. Random Forest was among the AI machine intelligence models used. In fault identification and correction (FDD), AI-based methods are all incredibly accurate, regardless of whether specific sensors are present or not [7].

Sofiane Kichou, Houcine Oudira, et al. in the next study. They diagnose solar power plants (PVs) and verify problem detection by utilising the random forest machine learning technique. Owing to the several problems that photovoltaic systems encounter, such as shading, open circuits, line-by-line issues, and short circuits, two random forest classifiers have been created: one for fault diagnosis (a multiclass classifier) and another for fault detection (a binary classifier). PV system performance is detected and tracked using random forest classification methods. The model has shown good results, with an error of the root mean square (RMSE) value of 0.0122 and 99.4% classification accuracy rates for fault diagnosis and detection [8].

Writing about their experiences, Paria Movahed, Saman Taheri, and Ali Razban present A framework for fault detection and analysis for bi-level data-driven of HVAC systems with machine learning algorithm. Energy consumption and maintenance are high for HVAC systems that aren't functioning correctly. They developed a second level machine learning architecture for monitoring problems in rooftop units (RTUs) and air handling units (AHUs) based on Principle Component Analysis (PCA), Random Forest (RF), and time series anomaly identification. They discovered that the model correctly detects eight different kinds of issues with rooftop units, multi-zone AHUs, and single-zone AHUs. A principal component analysis (PCA) uses one principal component to account for 95% of the variation in a dataset, therefore reducing its dimension. Additionally, random forest's accuracy was 89% for AHU in a single zone, 85% for AHU in a multi-zone, and 79% for RTU [9].

To evaluate the diagnostic and pinpoint a rooftop appliances shortcomings, Jalal Faraj, Amy Thompson, Mohammed G. Albayati, and others conducted a follow-up study using a semi-supervised machine learning approach. They used supervised algorithms, such as support vector machine (SVM), a combination of unsupervised algorithms, such as KNN and SVM, and semi-supervised algorithms, which combine support vector and KNN labelling to analyse fault detection, to address the issue of fault and maintain energy expenditures, servicing expenses, and enhance system dependability and customer happiness. They obtained an average accuracy of 93.5 for the SVM model, 94.9 for the combination of SVM and KNN, and finally, 94.9 for the combination of SVM, clustering, and unsupervised learning of k -NN. Labelling, they achieved the optimal accuracy of 95.7 when compared to the other two models.

Because of this, the third model is an effective strategy for handling a small number of labelled data points [10].

In the seventh study, Shuqing Wen, Yan Wang, Yifu Sun, Boju Huang, Shouguo Bian, Weirong Zhang, and Lin Zhao examined the issue of an improved principal component analysis approach with a clustering algorithm and Savitzky-Golay filter for sensor defect diagnosis and detection. Clustering and Principal component analysis approach were employed for analysis, and the results of their study were excellent. They identified and detected sensor faults using density-based spatial clustering. Also, the DBSCAN method automatically splits the dataset into smaller sub-datasets. Each smaller sub-dataset is then smoothed using the SG technique, and the dataset that has been processed is used to create a sub-PCA model, which eventually finds and diagnoses errors. Significant advancements in fault identification and detection have been made in the DBSCAN-SG-PCA model. This suggested model increases the Youden index on average is 0.21 while reducing the quantity of data needed for defect identification by an average of 13.7%. Moreover, the suggested model's accuracy in detecting faults is within 0.7 degrees Celsius. Once more, under various operating settings, the AHU system's working conditions were successfully determined by the DBSCAN approach [11].

More study has been conducted on the topic related to using a cascade neural network to find problems in solar-powered heating systems by Adolfo J. Sanchez, Sara Ruiz-Moreno, Eduardo F. Camacho, and Antonio J. Gallego. To detect three different types of faults, they first used a model-based fault detector. Subsequently, they utilised vibrant neural networks in cascade, self-organizing maps, random forests, classification trees, static neural networks and radial basis function networks, alongside and without decoupling plans. The model-based detector in their results achieved 90% accuracy, but the dynamic neural network model was their best model, with over 70% accuracy. Additionally, the ANNs combine functions in two layers: one layer categorises the error, and the other layer combines the data obtained from many classifiers, all of which have F1-scores greater than 50% [12].

An additional study on an ensemble-group-based flaws detection and diagnosis approach for assessing fault seriousness at the whole-building level was conducted by Akiluyanusa-Kaltungo and and Ashraf Alghanmi. The XGBoost classifier was utilised in this article to determine the building's energy performance. Additionally, they classified a number of architectural flaws using various ensemble multi-class classification techniques. Moreover, study is done on how noise affects training datasets. According to study, they used ensemble-based classification techniques, such as XGBoost, SKC, GBC, and RF, to pinpoint the underlying reasons for the unusual behaviour. Within 47 minutes of running the XGBoost model, they achieved 96 percent accuracy. Comparing the XGBoost model to other classifiers, it did well in terms of noise sensitivity [13].

Unsupervised learning techniques were employed in subsequent research by Zhiwei Ji, Ke Yan, Jing Huang, and Wen Shen for the diagnosis and fault finding of air conditioning systems. In order to solve the unbalanced data problem in FDD regarding air handling units (AHUs), they used generative adversarial networks (GANs) in their work. Following that, they trained the rebalanced datasets using supervised classifiers. includes multi-layer perception (MLP), RF, SVM, DT, and k-nearest neighbour (KNN) in unsupervised machine learning. For every defect category, there are 5, 10, 15,...,40 faulty training examples. As a result of using GAN, all of the fundamental classifier types' F-score, recall, accuracy, and precision values come in close to 1 [14].

Ronny Francis Ribeiro Junior, Isac Antônio dos Santos Areias, Mateus Mendes Campos, et al. used 1D multilayered neural networks using multiple interfaces vibration signals. They After employing hyper parameter tuning, they were able to achieve 99.92 percent accuracy in detecting and diagnosing six distinct types of problems in an electric engine through the use of several head 1D Convolution Neural Networks (1D CNNs). This method's extremely quick training and testing data set process is another advantage. For real-time applications, this technique is quite helpful for this reason [15].

The Random Forest machine learning algorithm was used in the following study by Radhia Fezai, Khaled Dhibi, Majdi Mansouri, et.al., to find wind energy conversion system fault detection. They employed random forest using the raw data set, but the noise in the data caused them to perform poorly. To solve this issue, they applied kernel principal component analysis. The acquired results demonstrate a high diagnosis accuracy of 91% and a low calculation time. Because of this, the random forest technique works well for fault detection in wind energy conversion systems [16].

Mario Mendoza and Pavel V. Tsvetkov conducted a follow-up study on the topic of unknown defect detection for reactor operational resilience. For anomaly detection, they employed reconstruction Error (RecErr), clustering (DBSCAN), and One-Class Support Vector Machine (OC-SVM); for dimension reduction, they used The Neural Network Autoencoder (DNNAE), convolutional Neural Network autoencoder (CNNAE), and Principal Component Analysis (PCA) in this study. Subsequently, they evaluated precision, recall, and F1 score to determine which model was the best and had the highest accuracy. The most accurate model is CNNAE-RecErr, which achieved 99.19 percent accuracy; the other two models, with different noise levels, obtained 98 and 97 percent accuracy, respectively [17].

Another study by Yingqian Liu, Rongyong Zhang, Zhaoming He, et al., examined how anomaly and fault detection are necessary for hydraulic machinery status monitoring. They proposed a complete framework built on support vector machines (SVM) and support vector data descriptions (SVDD) and employed the feature extraction approach known as TFF_VMD for this aim. They obtained satisfactory results for all of this, with anomaly detection accuracy for the SVDD model being 89, 99, and 99 percent, and for the SVM being 95, 96, and 98 percent, on three datasets [18].

In a further study, Ashraf Alghanmi and Akilu Yunusa-Kaltungo examined fault detection in an hot climate area of a public building. To find the building's defect, they employed random forest classifiers (RF) for this research. Apart from the exhaust fan, which was equipped with. an 82% accuracy rate, the RF classifier produced results with an accuracy rate of almost 90% and a high f1-score of more than 90% in the bulk of error cases [19].

Next research project: Ahmed Faris Amiri et al. completed photovoltaic (PV) systems analysis and fault detection approach using machine learning. For defect identification and diagnosis, they employed Random Forest classifiers in two distinct classes: one was binary, and the other was multiple. They obtained 99.4% classification accuracy rates for both fault detection and diagnosis, with an RMSE (root mean square error) of 0.0122. For increased accuracy and efficiency in machine learning, random forest is a promising model [20].

One additional study by Umair Sarwar and colleagues. They conducted an investigation of the turbine gas engine's defect diagnostics and detection. They combined artificial neural networks using data from multiple sensors to achieve this goal. To improve accuracy and expand the gas

turbine's utility, they first merge the principal component system with data from several sensors. Subsequently, they employed artificial neural network techniques such as multilayer perceptrons. With computed errors of 0.00173 and 1.9498, the hybrid model can identify the circumstances around a commercial gas turbine engine with good diagnostic accuracy. A comparative analysis of the two approaches shows that the supervised learning strategy is superior [21].

One more investigation by Milos Bjelic, et al., to identify the defect detection from the power transformer, they only employed machine learning algorithms. Sweep frequency response analysis is the monitoring basis for this study. To divide the data into groups with comparable states and failure probabilities, the machine learning method employed k-means clustering. Afterwards, they employed the Neurofuzzy Inference System (ANFIS) and Artificial Neural Network (ANN) to find the error. While the clustering approach is useful, it yields less accurate findings than supervised machine learning. In their result, they estimated 23 indicators with the use of SFR analysis. Excellent results are produced by the ANN and ANFIS methods, which can be applied to enhance asset management [22].

In a different study, Manoj Kumar Nigam and Debshree Bhattacharya employed hyperparameter tuning in a machine learning classifier to find the fault. They classified and detected faults include Light Gradient Boosting Machine (LGBM), XGBoost (XGB), Decision Tree (DT), and Random Forest (RF). To find the unbalanced data problem, they applied the SMOTE approach. Using the SMOTE approach, the resultant dataset was balanced, hence reducing the training time. The LGBM classifier had 100% detection, and classification accuracy increased by using the three feature selection strategies of Recursive Feature Elimination, Pearson co-relation, and univariate Feature (t-Test). Comparing the LGBM classifier with hyper parameter tuning to other ML models, it also produced results with excellent accuracy [23].

Another study on fault detection in PV solar arrays was subsequently conducted by S. Saravanan, R. Senthil Kumar, and P. Balakumar. They solved the reconfiguration mechanism of a partial shade PV array using the Binary Firefly Algorithm (BFA) technique to lessen the shadow's influence. Subsequently, physical degradation in PV panels is found using a Nave Bayes-based machine learning (ML) technique. Nave Bayes model got 99% accuracy over performed other models. Furthermore, the results of the BFA approach demonstrate that the suggested technique delivers high-power improvement and higher performance with the least amount of power loss [24].

Following that, in a different study, Ani Harish, Prince Asok, and Jayan M.V. employed machine learning techniques to identify and categorise transmission line defects using WAMS data. They did this by utilising multilayer extreme layer machines (ML-ELM) networks and auto encoder neural networks. To sum up, auto-encoder models identify errors with 100% accuracy and categorise them with 99.79% and 99.92% accuracy in 91 milliseconds. In within 29 milliseconds, the ML-ELM model was able to identify faults with 100% accuracy and classify different fault kinds with 99.92% accuracy. When compared to SAE models, ML-ELM models are generally faster computationally [25].

Furthermore, in a work by Md. Omaer Faruq Goniet al., they analysed fault identification and categorization in transmission lines with the application of an extreme learning machine. They divided the data into normal and faulty using MATLAB. For both faulty and normal data, our method effectively obtained 99.18% and 99.09% fault classification accuracy, as well as

99.53% and 99.60% fault detection accuracy. The extreme learning machine (ELM) model required less time than the artificial neural network (ANN) model. For fault detection, this approach outperforms an ANN by a factor of 7.6 [26].

The next study by Rajesh Natrajan et al. suggested employing machine learning technique for finding flaws in robotic autonomous control. in order to identify faults and emit states. They employed a Gaussian filter and a gradient decent based support vector machine (AGDSVM) in the automatic control system as a result. The combined system is known as (AGDSVM + GF). They obtained satisfactory results in terms of accuracy (98%), error rate (45%), state estimate (95.2%), error detection (97.5%), computation time (40s), and energy usage (47%). The study concludes that the combined approach is useful for analysing robotics failure detection [27].

Arezoo Dahesh et al. conducted a study in which they employed a hybrid machine learning model to analyse flaw identification in the wood industry. Nine different models were selected for this study: Random Forest (RF), Light Gradient Boosting Machine (LGBM), Adaptive Boosting (AB), Categorical Boosting (CAT), Extra Tree (ET), Gradient Boosting (GB), and Stacking with Multilayer Perceptron (MLP) meta-learner, in addition to Stacking with Support Vector Machine (SVM). After utilising each of these models, they used Bayesian hyper parameter optimisation techniques to evaluate the models. The CAT model displayed training and testing accuracies of 0.895 and 0.909, while the XGB model obtained training and testing accuracies of 0.884 and 0.885. Nonetheless, the RF model is the best overall model; its hyper parameters are changed using random search, and it is selected as the final model due to its stability, as seen by its testing and training accuracies of 0.905 and 0.893[28].

In the following work, machine learning algorithms were used to investigate finding defects and diagnosis in refrigeration systems by Zahra Soltani et al. This study compares the performance of linear discriminant analysis, classifiers using SVM, principal component analysis, convolutional neural networks, linear discriminant analysis, and support vector machines The SVM model has the best classification performance of all the models, with 95% classification accuracy with false positive rate 4%. Hence, SVM is the ideal model for identifying and detecting refrigeration faults [29].

Chapter third discuss in detail about methods which are used in this study. Before that data collection and exploration, data pre-processing, FCUs and types of faults present in FCUs these points are also discussed in next chapter.

Chapter 3: Methodology

The word "methodology" refers to the organised plan or series of actions used to conduct research, investigations, or problem-solving activities. It encompasses all of the methods, tools, and procedures used to gather, examine, assess, and draw conclusions from data or information. It provides validity, consistency, and dependability in the research or problem-solving process, assisting in the production of precise and significant results.

3.1 Data collection and Exploration:

For this study the data is collected from building which has many floors, and each floor has FCU unit. There are 731 FCUs unit and data is collected from them. Due to these reason data is separated in 731.csv files. Which has five attribute like derived, attribute name, id, date time and value. From all these columns only value column is numerical others are categorical.

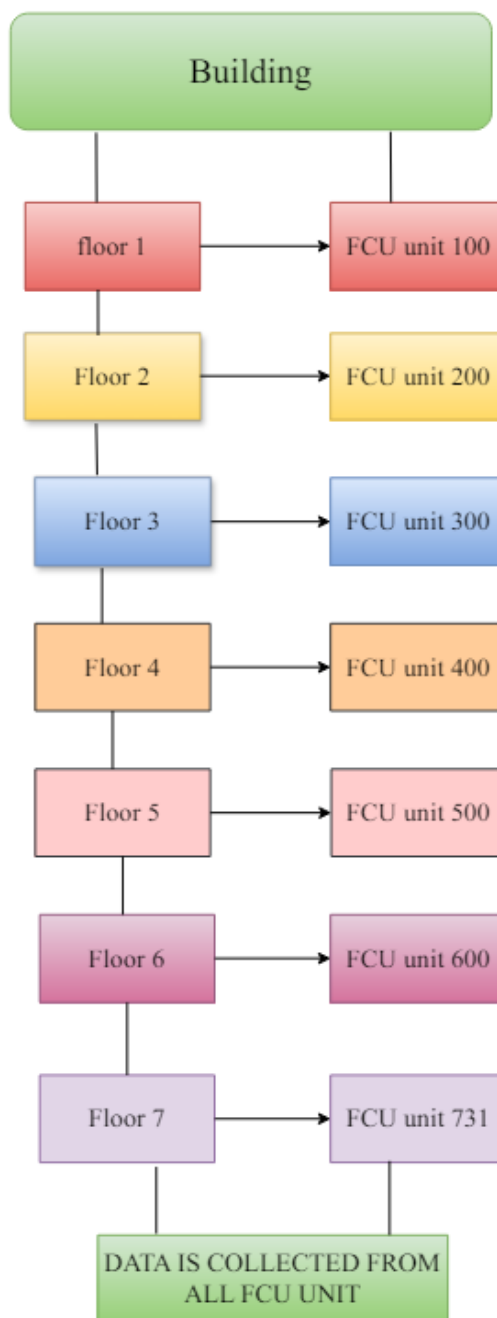


Figure 2: Diagrammatical representation of data.

3.2 FCUs unit:

One of the most essential parts of HVAC systems stand for heating, ventilation, and air conditioning frequently seen in buildings is a fan coil unit (FCU). An FCU basically consists of controls, a fan, and a heating or cooling coil. Its main purpose is to heat or cool the air that passes through a specific space in order to control the temperature and airflow within it. Due to their flexibility, FCUs can be deployed in a range of environments, such as commercial, industrial, and residential buildings. In order to provide localised temperature control and

increase energy economy and occupant comfort, they are frequently utilised in conjunction with centralised HVAC systems. Because FCUs are modular, they are easily involved into new construction projects or effortlessly incorporated into existing structures, which makes them a popular choice for HVAC systems in buildings of all sizes.

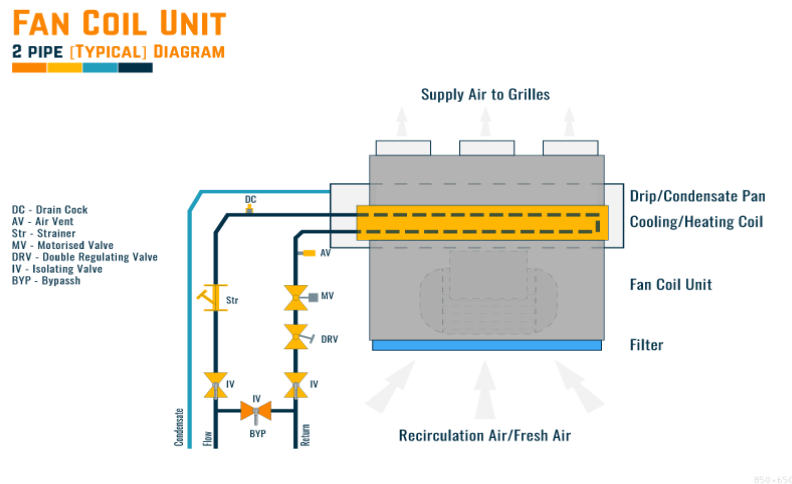


Figure 3: Typical FCU unit diagram [30].

3.3 Faults:

Building system faults, such as those affecting Fan Coil Units (FCUs), may hinder indoor comfort and interfere with HVAC systems' ability to operate efficiently. These malfunctions include a variety of difficulties, including insufficient cooling, restricted airflow, and excessive noise. Problems with compressors, sensors, or refrigerant leaks can all affect cooling effectiveness and result in insufficient temperature control. Airflow constraints can be caused by faulty fan motors, clogged filters, or blocked ducts, which prevents the air from properly circulating around the space. Uncomfortable and a sign of underlying mechanical problems, excessive noise is frequently the result of fan or motor breakdown. For FCUs to operate at their best and to keep an atmosphere that is comfortable indoors, these problems must be swiftly addressed through routine repair and maintenance.

3.4 Types of units and faults in FCU unit:

Fan coil units (FCUs), such as those used in heating and ventilation applications, are frequently unmonitored. As a result, system fan failure may go unnoticed until it becomes necessary to upgrade or refurbish the system or until the ventilation system stops producing air [31].

There are three types of FCUs unit basically in the residential areas. Vertical cabinet fan coils are either forced up in or near the wall or pressed against it. These units are not vented and usually only heat and cool one room. Fresh air can be provided up to 25% of the way through the wall at the back of the unit. Horizontal units installed on the ceiling are often piped on both the supply and return sides, if not both. The airspace above the ceiling serves as the return into a basic unit, and filters can be installed in a ceiling return plate. Vertical closet units are frequently used in utility closets, usually next to an outside balcony. Filters, a grilled return, and a vertical ducted supply are included with these systems.

Cooling performance problem, air flow and sound these are main three types of problem are mostly occurred in the FCUs unit of buildings [32].

Fan coil units are unable to tolerate more air resistance because of their squirrel cage fans. The air flow to resistance ratio is often designed to be significantly lower on many installations than the real resistance. Resistance is also increased by dirty coils, filters, and plenums in some systems. Increases in resistance lower air volume (CFM), which can cause an area to heat or cool irregularly.

Compressor water-cooling systems are required for large installations to distribute water at a specific temperature and volume across long distances, usually upstairs over many levels. Most fan coil systems use three or four row chilled water coils, which reduces their ability to make up for slight deficiencies in the chilled water loop. A greater temperature or a lower amount of cold water are frequently supplied.

For the house's heating and cooling requirements, whole systems and silent gadgets are required. Since everyone has a different range of hearing, what one person perceives as quiet may be loud to another. They must be as silent as possible because these gadgets will be working either in the actual space or right above it. Many installations overlook this, thus in order to get rid of irritating noise, owners or tenants will need to pest maintenance and HVAC suppliers nonstop.

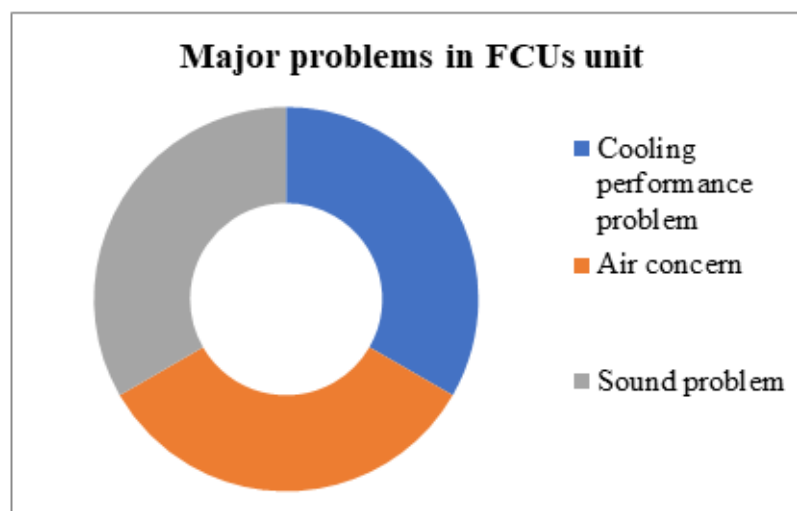


Figure 4:Diagrammatical representation types of faults in FCU unit.

3.5 Threshold:

A threshold is a predefined value in statistical analysis that is used as a standard or baseline for judgement calls. It acts as a border or cutoff point for grouping data into distinct categories or identifying whether particular qualities are present or absent. In several fields, such as machine learning, signal processing, risk management, and medical diagnostics, thresholds are used frequently.

For instance, a threshold is frequently employed in machine learning classification tasks to convert continuous prediction scores into binary results. The sample is categorised as belonging to one class if the prediction score is higher than the threshold; if not, it is classed as belonging to another class.

Here in the fault detection data the threshold which chosen for this analysis that is three according to control temperature in this data set the range of control temperature is from 21 to 24 so used threshold as three means if value beyond threshold it means faulty.

3.6 Data Preprocessing:

Data pre-processing is again divided into different subcategories like data cleaning and transformation, label encoder, one hot encoding etc.

3.6.1 Data cleaning and transformation:

This used data is for one building which have several floors and 731 FCUs. this data is collected from each of them on daily basis for analysis and diagnosis of fault.

Data cleaning is the process of removing duplicate, incomplete, incorrectly polluted, and badly formatted data from a dataset. Data duplication or incorrect labelling is a frequent occurrence when integrating many data sources. Moreover, Data transformation is the process of altering the format or organisation of a data set. Both data cleaning and transformation is important step for analysis.

In this dataset there are 731 data files in .CSV format which merged into one data frame for analysis purpose. After that there are five columns which have different type of information like attribute, id, date time and value that is control temperature.

After that, in data transformation the value column is converted into ten feature vectors. It is a method for extracting historical trends and relevant information from a series of data points. These feature vectors are useful because by taking use of the data's sequential structure, a ten-feature vector allows for a more reliable and precise problem detection method. For the analysis purpose the data are very huge so, it's better to change value column into feature vector so it's easy to analysis classification problems like SVM, KNN etc.

After that add label column using threshold concept. This label column is indicated whether each observation represent a faulty and non-faulty FCUs. Here in this method the threshold is consider as three according to control temperature it signifies a large deviation from the desired performance level or temperature. The values beyond threshold it indicates as faulty that is one otherwise non faulty zero.

After these pre-processing and transformation steps now, data is ready for analysis the fault detection in FCU unit.



Figure 5: Steps used for data cleaning and transformation.

3.6.2 Label encoder:

Label encoding transforms categorical information into numerical representations that machine learning systems can understand. Python label encoding can be implemented by using the Label Encoder class from the scikit-learn component. By adapting the encoder to the various categories in the data, it provides methods for converting the categories into numerical labels.

In fault detection the use of label encoder is to transform derived and attribute name column into numerical for using models like KNN or SVM for fault analysis.

3.7 Noise:

To avoid over fitting of the model noise is one of the best options to use and accurate the accuracy of model. Label noise is introduced to recreate imperfections or inconsistencies in the training labels, hence replicating faulty data collection or labelling processes. By using label noise, the model is taught to be more robust and tolerant of errors in the training data. This could prevent over fitting and improve the model's capacity to generalise to fresh data. It can be added to help evaluate the model's performance under less-than-ideal conditions and investigate its sensitivity to noisy labels.

Here used flipping noise this requires selecting a predefined percentage of labels at random and switching them to the opposite class. To be more specific, assigned random binary values (0 or 1) to the indices representing the fraction of samples to flip, and then flipped the labels at those indices. This method evaluates the model's performance and resilience in noisy situations by adding random label noise to the training set.

A binary random variable can be used to describe the mathematical equation that introduces label noise. 'ε' decides if a label is flipped or not.

For instance, y_i as the original label of the i -th sample and $\hat{y}_i$ as the noisy label of the i -th sample. Then equation of noise is:

$$1 - y_i \text{ with probability } \epsilon$$

$$\epsilon y_i \text{ with probability } 1 - \epsilon$$

For each label flip, the probability is represented by the noise level (ϵ). With probability ϵ , the label y_i is flipped (converted to the opposite class) with probability ϵ and $1 - \epsilon$, but the original label y_i remains untouched.

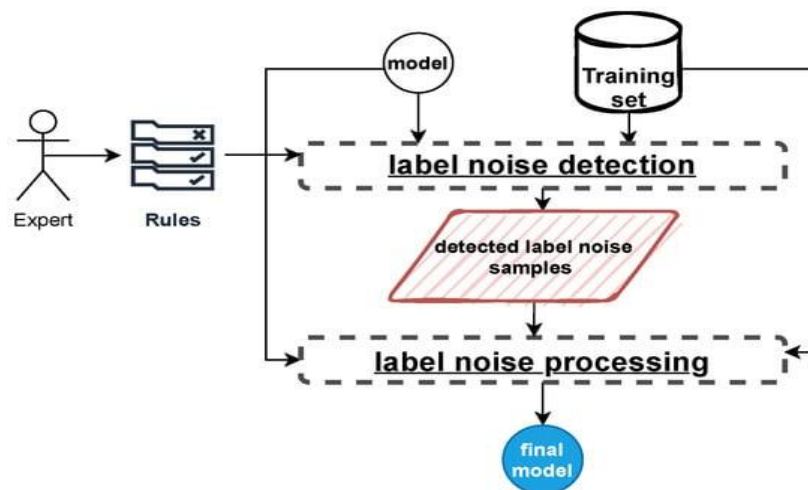


Figure 6: Diagrammatical representation of label noise (Q. Nguyen, Tomoaki Shikina, D. Teruya, S. Hotta, H.-D. Han, and Hironori Nakajo, “Leveraging Expert Knowledge for Label Noise Mitigation in Machine Learning,” *Applied sciences*, vol. 11, no. 22, pp. 11040–11040, Nov. 2021, doi: <https://doi.org/10.3390/app112211040>) [33].

3.8 Data availability and size:

The study presented a significant problem due to the data involved. The initial dataset included 731.csv files for a single month's worth of records, but not all of the files were defective. Therefore, 628 files with faults were obtained by applying the threshold approach based on control temperature; once more, transformed the value column into a ten-feature vector for analysis. However, because of memory problems, this analysis was only able to work on 100 files, after merge of 100 files data containing 16 columns of and 1048576 data points.

3.9 Machine learning algorithm:

An artificial intelligence (AI) system can perform tasks, such as identifying new data insights and patterns or forecasting output values based on a given set of input variables, by using a set of rules or procedures called machine learning algorithms. Learning in machine learning (ML) is enhanced by algorithms.

Deep learning is a specialised application of machine learning algorithms with advanced features. The difference is in how every algorithm learns. "Deep" machine learning models can use labelled datasets, they are called as supervised learning, to inform their algorithms, but they do not always require labelled data.

Several classification algorithms, such as support vector machine (SVM), k-nearest neighbours (KNN), deep multi-layer perceptron (Deep MLP) and multi-layer perceptron (MLP), are utilised in this study to find faults in FCUs.

3.9.1 Support vector machine:

In an n-dimensional feature space, each data point is represented as a feature vector (x_i) with n attributes. Data points are labelled with a class label (y_i)(e.g., normal or faulty) to indicate their classification. SVM tries to locate the best hyper plane that separates data points from distinct classes by the greatest margin. The hyper plane is described by the equation:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

where, $\mathbf{w}$ = weight vector that is not parallel to the hyper-plane

$\mathbf{x}$ = input vector of features, and

b = bias expression.

SVM maximises the gap between each class's (i.e faulty and non-faulty) hyper plane and its nearest data points (support vectors). The margin is calculated as the distance between the hyper plane and the support vectors. Moreover, it solves an optimisation issue to discover the best hyper plane by maximising the margin and minimising the classification error. This is commonly described as a constrained optimisation problem. In fault detection, SVM can be trained on labelled data containing both normal and faulty cases to learn the properties of each type. Once trained, SVM may use feature vectors to determine if new data points are normal or incorrect. Here in this case use input as label then pre-process the numeric feature. Used linear and SGD kernel for optimization. Lastly, evaluate the SVM model using precision, recall and confusion matrix. In some cases, to avoid over fitting of model label noise term is introduced in SVM.

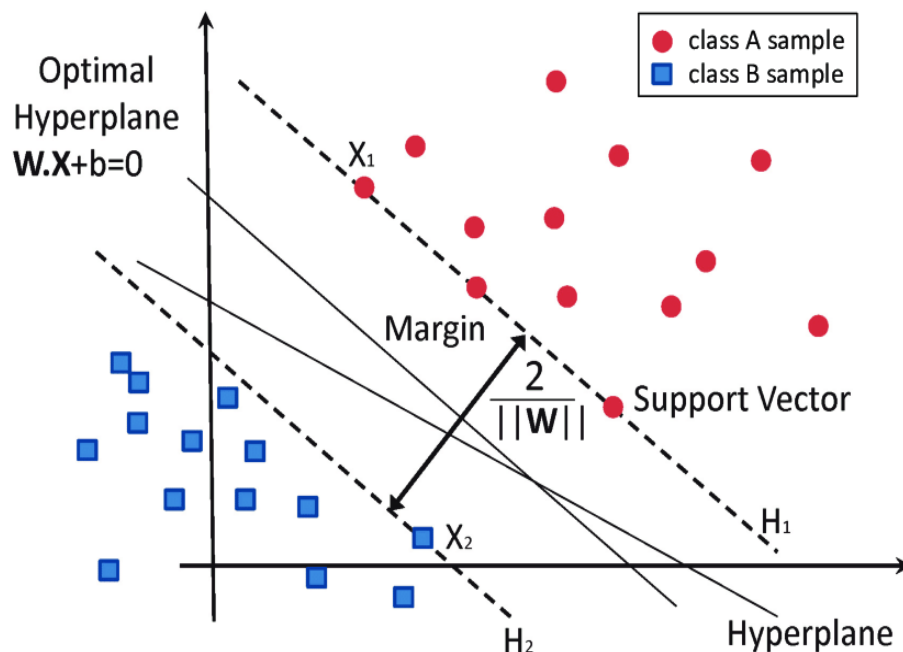


Figure 7:SVM operation using a diagram "Data classification using support vector machines (SVM)" in Figure 2. Classification of data by support vector machine (SVM) fig8_304611323, ResearchGate, <https://www.researchgate.net/figure/> [34].

3.9.2 K-Nearest Neighbors (KNN):

The fault identification technique K-Nearest Neighbours (KNN) is simple yet efficient. Every data point in the training set has its feature vectors and labels tracked by the model. In order to predict a new data point with an unidentified label, KNN locates the k nearest neighbours with a distance metric in the feature space, most frequently the Euclidean distance. The distance between each point and new point in the training set is calculated. The k nearest neighbours is calculated by selecting the k data points having the shortest distances from the new point. Once the nearest neighbours have been located, the method assigns the majority class to the new data point.

KNN can be used in fault detection scenarios by transforming each data point into a feature vector containing sensor readings, functional variables, or other relevant metrics. The system can then discover patterns in the data and detect departures from usual habits, which may indicate defects or anomalies.

Moreover, for fault detection tasks, K-Nearest Neighbours (KNN) is a straightforward yet powerful method. KNN classifies data points in defect detection scenarios according to the majority class of the nearest neighbours. In fault detection terms, this indicates that if the majority of a data point's closest neighbours are also categorised as belonging to the same fault category, then the data point is also classed as belonging to that category.

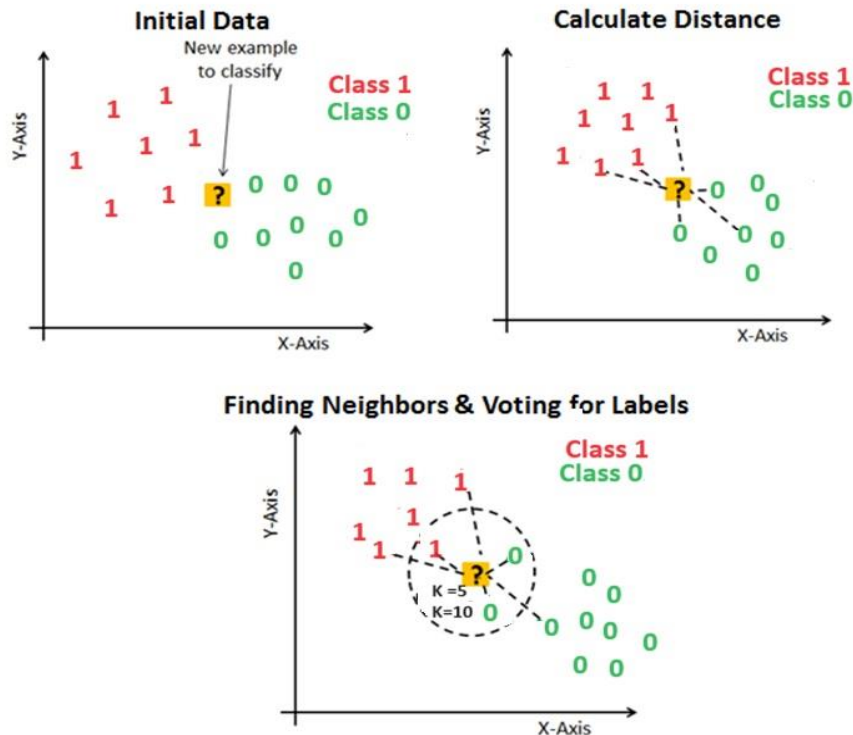


Figure 8: diagrammatical representation of how KNN works.

3.9.3 Multilayer Perceptron (MLP):

An (ANN) artificial neural network is called a Multilayer Perceptron (MLP) made up of many layers of entangled neurons, consisting of an output layer, an input layer, and one or more hidden layers. The input layer is made up of neurons that represent the features in the incoming data. Each neuron represents a single feature, and its values serve as input values to the network. Hidden layers are made up of neurons that perform nonlinear modifications on input data. After calculating the weighted total of all the inputs from all the neurons in the preceding layer, each neuron in a hidden layer receives information about those neurons and applies an activation function to them. The ultimate output of the network is produced by neurons in the outer layer. The last layer's total neuron count fluctuates depending on the goal.

While training, the network uses optimisation algorithms like gradient descent for a loss function reduction that assesses the difference between forecasted and true labels. Dropout, weight decay, and early stopping techniques can all help to reduce over-fitting and increase generalisation performance. MLPs are sophisticated models capable of learning complicated correlations in data, making them useful for any number of tasks, including fault detection.

To automatically learn and identify anomalies or flaws in a system, Multilayer Perceptron (MLP) models are used in fault detection. These models work by collecting information from different sensors or metrics related to the system's performance. The MLP optimises its performance to correctly categorise instances as either faulty or normal during training by iteratively adjusting its internal parameters based on a labelled dataset. Fault detection tasks can benefit from the use of MLP models since they are effective at identifying hidden trends in complex datasets and capturing nonlinear relationships.

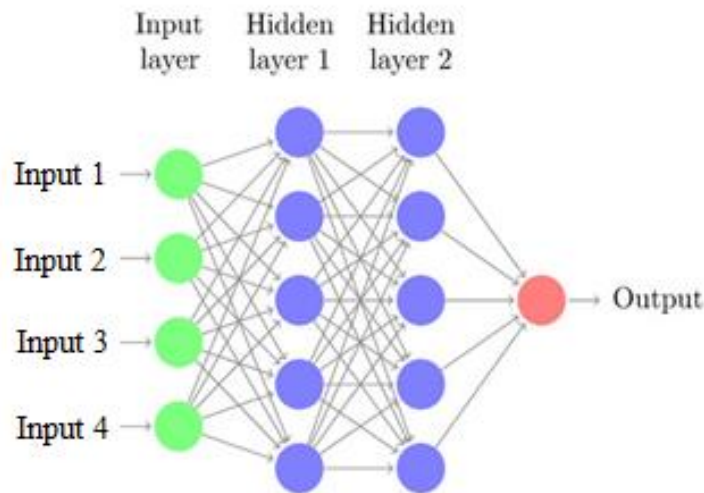


Figure 9: Diagrammatical representation of MLP model with two hidden layers.

3.9.4 Deep Multilayer Perceptron (Deep MLP):

A Deep Multilayer Perceptron (Deep MLP) functions similarly to a conventional MLP but has more hidden layers, allowing it to learn more complicated representations of the input data. Similar to a standard MLP, the neurons made up of input layer that reflect the input data's attributes. A deep MLP consists of numerous hidden layers, each of which implements nonlinear modifications on input data. The network's final predictions are produced by the output layer, which functions similarly to a normal MLP.

A deep MLP may learn hierarchical representations of the input data, capturing complicated patterns and correlations which cover numerous layers. Because of its capacity to learn complicated features, deep MLPs are well-suited to jobs that require high-level feature extraction, such as fault identification in sensor data. A deep MLP may detect defects based on data patterns by using past sensor data as input features and binary labels indicating either normal or abnormal operation.

Within DEEP MLP the target variable shows if a data point indicates a fault or not. The dataset contains a variety of numerical features. Five hidden layers, ranging in size from 128 neurons in the first hidden layer to 8 neurons in the last, make up the MLP classifier. To detect faults in a system, the deep MLP classifier learns intricate patterns and relationships from the input characteristics in order to differentiate between normal and abnormal system behaviour. Because of its depth, the MLP is able to recognise small defect indications by capturing complex and hierarchical representations of the input data. In general, the deep MLP classifier is an efficient tool for fault detection because it can learn from complex data and adjust to noisy settings, which helps it find abnormalities and faults in systems.

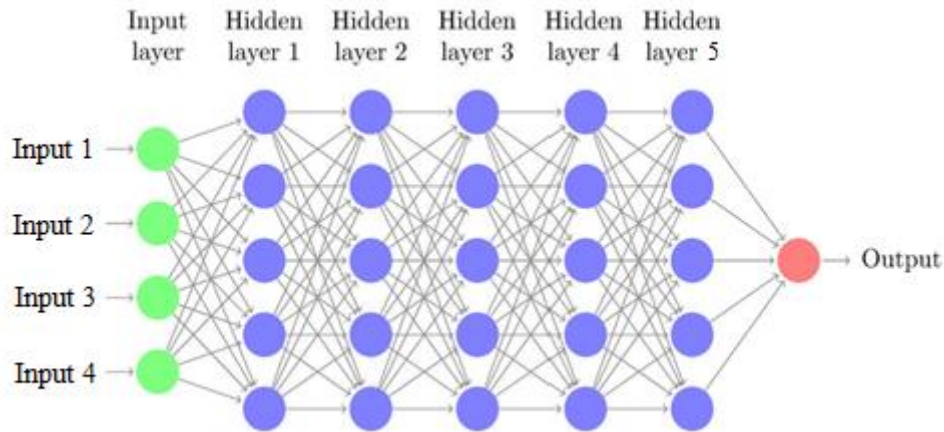


Figure 10: Working of DEEP MLP model with five hidden layers.

3.10 Model Evaluation:

After applying all the model, the next crucial step is to evaluation of model. Here for SVM, KNN, MLP and DEEP MLP the evaluation is based on confusion matrix, accuracy, memory, and F1 score, among others. The True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN) metrics constitute the basis for their derivation. Positive class members are TP and FP, while negative class members are TN and FN. The following describes the individual calculations of the confusion matrix, precision, recall, and f1 score:

3.10.1 Confusion Matrix:

A table called a confusion matrix is frequently used to describe how well a classification model works with a set of test data. The four components of a confusion matrix are False Positive (FP), True Negative (TN), True Positive (TP), and False Negative (FN). It gives an overview of the actual and expected class labels.

	Predicted classes	
Actual classes	0	1
0	TP	FP
1	FN	TN

Table 1: General overview of confusion matrix.

3.10.2 Precision:

Precision in the context of fault detection refers to a model's capacity to detect real problems (positives) accurately while reducing false alarms or false positives. Because it measures the system's dependability in identifying real faults without sounding false alarms in the event of typical system operation, precision is an essential statistic in fault detection systems.

$$\text{Precision} = \frac{TP}{FP} + TP$$

3.10.3 Recall:

In the area of fault, recall is frequently referred to as sensitivity or true positive rate. detection, quantifies a model's capacity to accurately identify every case of a real fault (positives) out of

every instance consisting of faults. The model's recall gauges its ability to spot real positives while reducing false negatives, or situations in which errors exist but are missed by the model.

$$\text{Recall} = \frac{TP}{FN} + TP$$

3.10.4 F1 Score:

Recall and precision harmonic means are represented by the F1-score. Between recall and precision, it finds a balance.

$$\text{F1 Score} = \frac{2 \times (\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})}$$

All these model evaluation techniques are applied on different models like support vector machine and k- nearest neighbour etc. Here used these model evaluation parameters for fault detection.

The fourth chapter discusses the use of Power Bi tool and Python to visualise data points and attributes.

Chapter 4 Analysis and Design

4.1 Data Description:

This data is for one building which has several floors and 731 FCU unit so data is collected from each unit on daily basis. But each file doesn't have fault because this data is deal with control temperature. According to threshold can find anomaly. So, after pre-processing data got 628 files which has chances to present fault. There are five columns each column is described below:

Variable	Description
Derived	This is name of source from value comes
Attr_name	This indicates name or attribute of the temperature
ID	It gives information about FCU unit unique id
Datetime	This column provides meaningful information about time when temperature measure
Value	It provides major source of temperature value.

Table 2: Description of each attribute present in dataset.

4.2 Exploratory analysis:

For this project, exploratory data analysis (EDA) involves investigating data's features and trends in order to obtain knowledge and guide. The main feature of EDA is visualisations, which allow to use box plots, density plots, and histograms to examine the distributions of individual variables. The dispersion, skewness, and existence of outliers in your data are all revealed by these visualisations. Understanding the underlying patterns and dependencies in the data also requires investigating the interactions between variables. Methods like correlation analysis, pair plots, and scatter plots are useful for identifying possible multi co-linearity problems and revealing relationships. To evaluate class imbalance as well as understand the

distribution of classes, it is also essential to look at the target variable(s) distribution, particularly in classification tasks.

4.2.1 Histogram:

The 'value' column's histogram offers information on the distribution of values in the dataset. The x-axis in this histogram displays the value range, while the y-axis indicates the frequency with which those values occur. Each bar in this below histogram reflects a set of numbers known as a bin, and the height of each bar shows how frequently values fall within that bin. By examining this histogram, we can see the central tendency, spread, and shape of the values in the 'value' column. In the below figure the value is ranging from zero to thirty because control temperature is mostly ranging from twenty-one to twenty four and deadband value is also three. Moreover, can say that this histogram is showing right skewed.

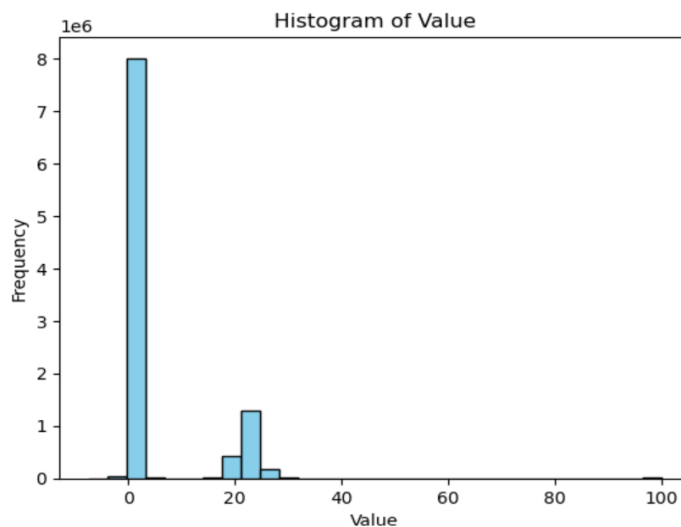


Figure 11: Histogram of value column.

4.2.2 Count Plot:

For every label, the number of data points is represented by the height of the bar. More data points are linked to that label with taller bars, and fewer data points are linked with shorter bars. It is clear from the visualisation below that label zero has a taller bar than label one. In other words, there are several instances of label zero. This graphic indicates that there is an imbalance in the dataset between two labels. Using concept threshold faulty means (1) and non-faulty means (0). Actually, zero labels data points are 838,769 and one labels data points are 209,806 out of a total of 1,048,575 labels.

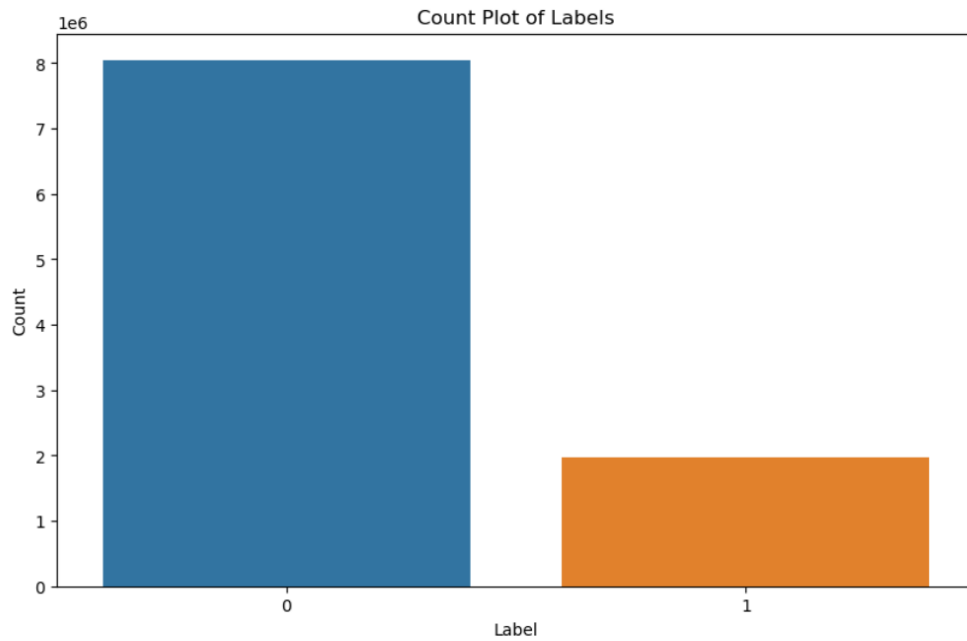


Figure 12: Count plot for labels.

4.2.3 Heatmap:

Each cell in the heatmap indicates the degree of association between two characteristics, and the colours denote the direction and strength of the link. When one feature rises, the other also tends to rise; a positive correlation is shown by colours in red. So according to below heatmap there is strong co-relation between value and each bin of value column. And the label value has 0.91 it means also label column has strong relation with value and all value bins.

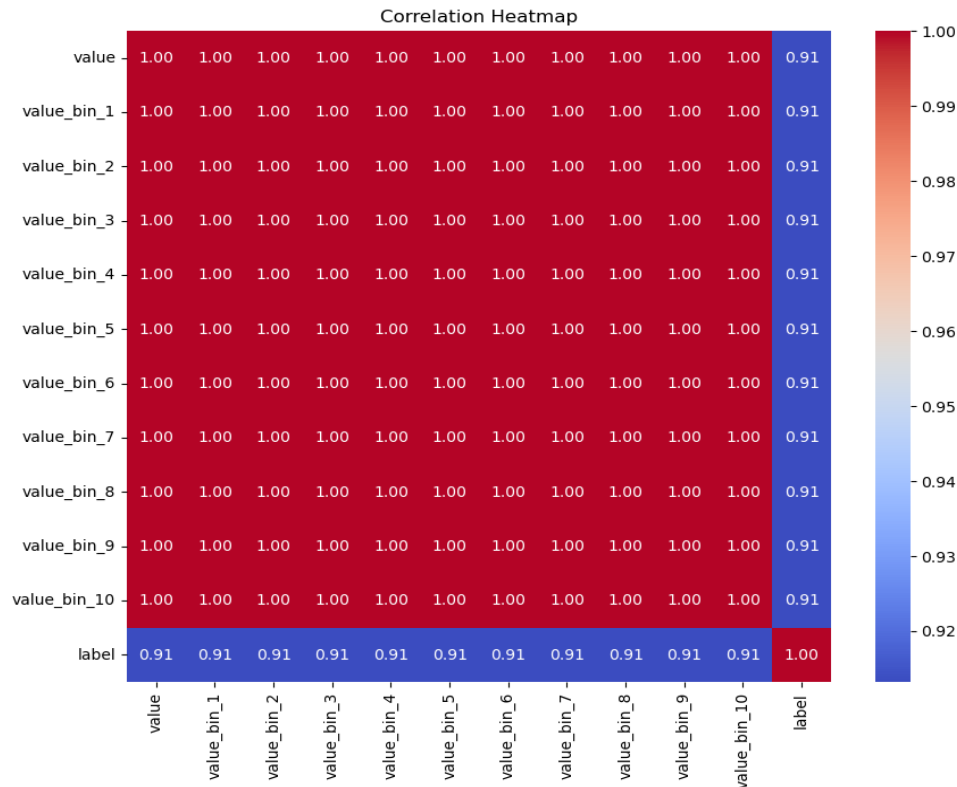


Figure 13: overall correlation heatmap for all numeric columns.

4.2.4 Count plot:

In the count plot on the x- axis there is attribute name and on y- axis the is count each bar in this below graph represent the attribute present in the data and height of each bar indicate the number of appearances in the dataset. From the below graph is_on, power, is_saturated has highest bar height as compared to others it means these attributes are less frequent in data set on the other hand the all attribute which has height close to 0.3 all have moderate height these attributes are useful for analysis so, in this analysis more focus on control temperature because this is the main source of fault detection.

1. **Slicer:** used to slice data with their attribute name. Slicer is important tool to show all data information of each attribute. And it easy to see the one attribute information like control temperature if clicked on that this desktop will give all information about control temperature.
2. **stacked column chart:** it shows how values are distributed and compared among several parts or categories. For example, cooling set point trace has higher value and is heating and cooling has lower value in this diagram. In the conclusion this chart is useful to gain insight between attribute name and value.
3. **Pie chart:** it shows the contribution of sum of value by label. There are two labels 0 and 1 that is non faulty and faulty. In this diagram label 1 has more contribution of value and 0 has less contribution of value. This chart is helpful for seeing patterns or trends and learning more about the dataset's makeup.
4. **Line chart:** this chart is form between sum of value and date time. This chart gives information about pattern and trends of value column over time. This trends in line chart can influence on the specific date and time. This chart provides future forecast of value by time.

The results and analysis are covered in the chapter five. All applied model results and their evaluations is discussed in next chapter.

Chapter 5: Result and Analysis

In this section results are from SVM, KNN, MLP & DEEP MLP with different types like with noise and without noise plus different percentage of train test split like 80-20% and 70-30% split as well as with different kernel and optimiser.

5.1 Result of KNN:

5.1.1 KNN without noise with 70-30%:

K-nearest neighbour was used to diagnose and detect faults by dividing the data set into 70% training and 30% testing in order to improve results. In this case, n equals 10. Following that, the KNN was evaluated using the confusion, recall, and precision matrices. The outcomes are listed below:

KNN 70-30% split without noise			
Training Accuracy		0.99	
Testing Accuracy		0.99	
Number of N		10	
Training precision	0.99	Testing precision	0.99
Training recall	0.99	Testing recall	0.99
Training f1 score	0.99	Testing f1 score	0.99

Table 3: Accuracy table for KNN with 70-30% split without noise

Training Confusion Matrix			
		Predicted class	
Actual		0	1
class	0	5634344	41
	1	124	1377396

Table 4: Training confusion matrix for KNN with 70-30% split without noise

Testing Confusion Matrix			
		Predicted class	
Actual		0	1
class	0	2414636	23
	1	62	590382

Table 5: Testing confusion matrix for KNN with 70-30% split without noise

Above table shows that for both training and testing datasets, the KNN (K-Nearest Neighbours) results show extremely good accuracy and performance metrics. The KNN model appears to have grasped the underlying patterns in the data, as evidenced by its 99% training and 99% testing accuracy, and its effective generalisation to new data. Moreover, at almost 99% for both training and testing, the model's precision, recall, and F1 scores are very good, showing that it makes a large percentage of accurate predictions while reducing false positives and false negatives. This shows that the model can detect both positive and negative cases in the dataset with a high degree of accuracy and reliability. In case of confusion matrix, the model's efficacy and robustness are further confirmed by the minimal quantity of false negatives and positives.

5.1.2 KNN model with 80-20% split without Noise:

Here using KNN model for fault detection with 80 and 20 percent training and testing data. Moreover, here the value of N is 10. Here changed the percentage of training and testing may give better result. The result table are below.

KNN 80-20% split without noise			
Training Accuracy		0.99	
Testing Accuracy		0.99	
Number of N		10	
Training precision	0.99	Testing precision	0.99
Training recall	0.99	Testing recall	0.99
Training f1 score	0.99	Testing f1 score	0.99

Table 6: Accuracy table for KNN 80-20 split without noise

Training Confusion Matrix			
		Predicted class	
Actual		0	1
class	0	5634344	41
	1	124	1377396

Table 7: Training confusion matrix for KNN 80-20 split without noise

Testing Confusion Matrix			
		Predicted class	
Actual		0	1
class	0	2414636	23
	1	62	590382

Table 8: Testing confusion matrix for KNN with 80-20 split without noise.

Here is also the f1 score, recall, precision and accuracy are same as above model. This model also learns underlying pattern and got high accuracy. But in confusion matrix the greatest number of predictions in the training and testing confusion matrices lie on the diagonal, signifying precise projections, while the off-diagonal parts indicate inaccurate predictions. The model's efficacy and robustness are further confirmed by the low number of false positives and negatives. Overall, model is good on both training and testing dataset.

5.1.3 KNN model with 70-30% split with Noise:

In above two types of splitting data are looks like over fitting so here used the term label noise to avoid over fitting and getting appropriate result. In this model 70 percent training and 30 percent testing data is used for analysis purpose and 0.5 is the label noise, moreover, number of N is same as above.

KNN 30-70% split with noise			
Testing Accuracy		0.9688	
Training Accuracy		0.969	
Number of N		10	
Noise		0.5	
Training precision	0.9202	Testing Precision	0.9206
Training recall	0.9213	Testing recall	0.9217
Training f1 score	0.9208	Testing f1 score	0.9212

Table 9: Accuracy table for KNN with 70-30 split using noise.

Training Confusion Matrix			
		Predicted class	
Actual		0	1
class	0	5524431	109954
	1	108323	1269197

Table 10: Training confusion matrix for KNN with 70-30 split using noise.

Testing Confusion Matrix			
		Predicted class	
Actual		0	1
class	0	2367783	46876
	1	46205	544239

Table 11: Testing confusion matrix for KNN with 70-30 split using noise.

As expected, given the addition of noise to the dataset, this model shows somewhat not too much good performance than the noise-free model. In spite of this, the model continues to show comparatively high efficacy and accuracy in the testing and training datasets.

Even with noise, the model retains a high degree of accuracy, with testing and training accuracy of about 96%. This shows that even in situations when the data is noisy, the KNN algorithm is robust and still capable of producing good results. The model's performance can be more clearly understood by examining the measurements of F1 score, precision, and recall. Even while the precision, recall, and F1 scores are marginally lower than in the noise-free case, they are still rather high, at approximately 92%. This suggests that the model maximises true positives while keeping an appropriate balance between minimising false positives and false negatives.

Overall, the findings show that the KNN method continues to work well and retains a high precision, accuracy, recall, and F1 score, even though the addition of noise has a minor effect on the model's performance. This highlights the algorithm's ability to handle noisy datasets with robustness and efficacy, which means it's appropriate for classification tasks within real-world scenarios.

5.1.4 KNN with 80-20 split with Noise:

Here also used label noise with 80-20 split of training and testing data set.

KNN 80-20% split with noise			
Testing Accuracy		0.9688	
Training Accuracy		0.969	
Number of N		10	
Noise		0.5	
Training precision	0.9202	Testing Precision	0.9206
Training recall	0.9213	Testing recall	0.9217
Training f1 score	0.9208	Testing f1 score	0.9212

Table 12: Accuracy table for KNN with 80-20 split using noise.

Training Confusion Matrix			
Predicted class			
Actual		0	1
class	0	5524431	109954
	1	108323	1269197

Table 13: Training confusion matrix for KNN with 80-20 split using noise.

Testing Confusion Matrix			
Predicted class			
Actual		0	1
class	0	2367783	46876
	1	46205	544239

Table 14: Testing confusion matrix for KNN with 80-20 split using noise.

The 80-20% split with noise KNN (K-Nearest Neighbours) findings show comparable performance to the 30-70% split case. The model retains good levels of recall, accuracy, precision, and F1 score in dataset which has both the training and testing over the introduction of noise into the dataset. When the precision, recall, and F1 score metrics are examined, they show that they are still quite high at about 92%, suggesting that the model successfully maintains a balance between maximising true positives and minimising false negatives and false positives.

The model is resilient to noisy data since it retains a low number of misclassifications in spite of the noise. All things considered, the findings indicate that the KNN algorithm functions admirably and keeps up high standards of efficacy and accuracy even in situations when there is noise in the data and varied train-test split ratios.

5.1.5 KNN with 70-30% split without noise (N=5):

In this model there is split is 70 percent training data and 30 percent testing without label noise. But, here the number of K is reduced from 10 to 5 because reducing the number of neighbours makes the model simpler and more responsive to small-scale fluctuations in the information. Reducing the number of neighbours usually results in increased variance and decreased bias.

When predicting a new data point, the number of neighbouring data points (N) in a KNN affects how many of them are taken into account.

KNN 70-30% split without noise			
Training Accuracy		0.99	
Testing Accuracy		0.99	
Number of N		5	
Training precision	0.99	Testing precision	0.99
Training recall	0.99	Testing recall	0.99
Training f1 score	0.99	Testing f1 score	0.99

Table 15: Accuracy table for KNN with 70-30 split without noise (N=5)

Training Confusion Matrix			
Predicted class			
Actual		0	1
class	0	5634345	40
	1	66	1377454

Table 16: Training confusion matrix for KNN with 70-30 split without noise (N=5).

Testing Confusion Matrix			
Predicted class			
Actual		0	1
class	0	2414630	29
	1	32	590412

Table 17: Testing confusion matrix for KNN with 70-30 split without noise (N=5).

Across a range of evaluation measures, the KNN model with five neighbours (N=5) and a 70–30% split without noise performs exceptionally well. With testing and training accuracies reported at accuracy 99%. A minimal percentage of false negatives and false positives is indicated by precision and recall values of 99%. The model's robustness in capturing true positives while minimising misclassifications is highlighted by the F1 score, which sits at 99% and balances precision and recall. Examining the confusion matrices shows how well the model classifies situations. High counts of true positives and true negatives are the outcome of most cases in both training and testing sets being correctly predicted.

Overall, the 5-neighbor KNN model performs exceptionally well, demonstrating its ability to accurately forecast and generalise to new data. These outcomes highlight the model's superior performance in completing the classification task has a great level of precision, accuracy, and recall.

5.1.6 KNN with 80-20% split without noise (N=5):

Here using 80-20 split with 5 number of N. The results are as follow:

KNN 80-20% split without noise			
Training Accuracy		0.99	
Testing Accuracy		0.99	
Number of N		5	
Training precision	0.99	Testing precision	0.99
Training recall	0.99	Testing recall	0.99
Training f1 score	0.99	Testing f1 score	0.99

Table 18: Accuracy table for KNN with 80-20 split without noise (N=5).

Training Confusion Matrix			
Predicted class			
Actual		0	1
class	0	5634345	40
	1	66	1377454

Table 19: Training confusion matrix for KNN with 80-20 split without noise (N=5).

Testing Confusion Matrix			
Predicted class			
Actual		0	1
class	0	2414630	29
	1	32	590412

Table 20: Testing confusion matrix for KNN with 80-20 split without noise (N=5).

In this model which use 5 neighbours (N=5) the model performs excellently on a number of assessment criteria. With testing and training accuracies reported at an outstanding 99%, the model is clearly able to classify most cases in both datasets with accuracy. In confusion matrix scenario there are a lot of true positives and true negatives because the model predicts most cases correctly. Further confirming the model's dependability is the low quantity of false negatives and positives, which show that the model can still clearly distinguish between the true positive and false negative. Overall, the KNN model is good choice for fault detection.

5.1.7 KNN with 70-30% split with noise (N=5):

Here using N number five with 70-30 split. Let's see the result below.

KNN 70-30% split with noise			
Testing Accuracy		0.897	
Training Accuracy		0.896	
Number of N		5	
Noise		0.5	
Training precision	0.99	Testing Precision	0.99
Training recall	0.99	Testing recall	0.99
Training f1 score	0.99	Testing f1 score	0.99

Table 21: Accuracy matrix for KNN with 70-30 split using noise (N=5)

Training Confusion Matrix			
Predicted class			
Actual		0	1
class	0	5634345	40
	1	66	1377454

Table 22: Training confusion matrix for KNN with 70-30 split using noise (N=5).

Testing Confusion Matrix			
Predicted class			
Actual		0	1
class	0	2414630	29
	1	32	590412

Table 23: Testing confusion matrix for KNN with 70-30 split using noise (N=5).

Considering that noise is present, this model performs excellently. In comparison to the noiseless scenario, the testing accuracy drops somewhat to 89.7%, although it still remains good. This demonstrates the model's resilience and ability to manage noisy data. The confusion matrix's training and testing sets have a high proportion of true positives and true negatives. The low rates of false positives and false negatives demonstrate that, even in the presence of noise, the model is still capable of effectively differentiating between the two groups.

In conclusion, the KNN model with five neighbours demonstrates its resilience and efficacy in classification tasks by performing perfectly with noise.

5.1.8 KNN with 80-20% split with noise (N=5):

0.5 noise included with 80-20 splitting the results are below.

KNN 80-20% split with noise			
Testing Accuracy		0.8967	
Training Accuracy		0.8963	
Number of N		5	
Noise		0.5	
Training precision	0.6794	Testing Precision	0.681
Training recall	0.897	Testing recall	0.8966
Training f1 score	0.7732	Testing f1 score	0.77408

Table 24: Accuracy table for KNN with 80-20 split using noise (N=5)

Training Confusion Matrix			
Predicted class			
Actual		0	1
class	0	5773747	665963
	1	162076	1411820

Table 25: Training confusion matrix for KNN with 80-20 split using noise (N=5).

Testing Confusion Matrix			
Predicted class			
Actual		0	1
class	0	1443825	165509
	1	40731	353337

Table 26: Testing confusion matrix for KNN with 80-20 split using noise (N=5).

Performance of this model is noticeably lower than in the noiseless scenario. There is a minor drop in comparison to the noiseless casing, even if the testing accuracy is still rather excellent at 89.67%. This suggests that noise introduces a little amount of variation into the model's capacity to generalise to new data. A similar pattern can be seen when looking at the measures for precision, recall, and F1 score. The equivalent metrics for the testing dataset are slightly lower at 68.1%, 89.66%, and 77.408%, respectively, despite the fact that the training precision, recall, and F1 score are quite high at 67.94%, 89.7%, and 77.32%, respectively. This indicates that, in comparison to the training dataset, the model's performance on the testing dataset is slightly restricted, most likely as a result of noise.

In conclusion, the KNN model that was trained on the noise-filled 80–20% split dataset performs rather well, although noise has a small impact. There is an apparent decline in comparison to the noiseless situation, even though it still retains a reasonably high accuracy and performs well in terms of recall, precision and F1 score.

5.2 Result of SVM:

5.2.1 SVM with 70-30% without noise:

Support vector machine algorithm used for fault detection with 70 percent of training and 30 % of testing data to see the result because splitting data is crucial part for machine learning result.

SVM 70-30% split without noise			
Training Accuracy		0.99	
Testing Accuracy		0.99	
Kernel		Linear	
Training precision	0.99	Testing precision	0.99
Training recall	0.99	Testing recall	0.99
Training f1 score	0.99	Testing f1 score	0.99

Table 27: Accuracy table for SVM with 70-30 split without noise

Training Confusion Matrix			
Predicted class			
Actual		0	1
class	0	5634345	40
	1	66	1377454

Table 28: Training confusion matrix for 70-30 split without noise.

Testing Confusion Matrix			
Predicted class			
Actual		0	1
class	0	2414630	29
	1	32	590412

Table 29: Testing confusion matrix for 70-30 split without noise.

In this model both training and testing accuracy is 99% so it shows that model effectively classify both classes. In case of evaluation these evaluation measures show that the model is flexible, correctly identifying both positive and negative cases, and maintaining a good balance between precision and recall. Upon examining the confusion matrices, it can be observed that the model accurately categorises most cases with very few misclassifications.

It can be concluded that the SVM model performs exceptionally well on a variety of assessment measures. It was trained using a linear kernel on a without noise 70-30% split dataset. The model appears to be quite successful in classifying instances and has good generalisation to new data based on its high accuracy, precision, recall, and F1 score metrics as well as the low misclassifications shown by the confusion matrices.

5.2.2 SVM with 80-20% without noise:

In different way using SVM model with 80-20 split. The results are as follows:

SVM 80-20% split without noise			
Training Accuracy		0.99	
Testing Accuracy		0.99	
Kernel		Linear	
Training precision	0.99	Testing precision	0.99
Training recall	0.99	Testing recall	0.99
Training f1 score	0.99	Testing f1 score	0.99

Table 30: Accuracy table for SVM with 80-20 split without noise.

Training Confusion Matrix			
		Predicted class	
Actual		0	1
class	0	5634345	40
	1	66	1377454

Table 31: Training confusion matrix for SVM with 80-20 split without noise.

Testing Confusion Matrix			
		Predicted class	
Actual		0	1
class	0	2414630	29
	1	32	590412

Table 32: Testing confusion matrix for SVM with 80-20 split without noise.

This result which shown in above table are same as the 70-30% split. It means that SVM with linear kernel which act same on both 70 % and 80 % training data.

5.2.3 SVM with 70-30 split with Noise:

In this model the new concept is introduced here that name called as label noise because from above two splitting of SVM 70-30 and 80-20 without noise both results are same that is 99 %. Somewhere this model shows over fitting to avoid this used label noise in this model. Let's see the result of SVM with label noise is below:

SVM 70-30% split with noise			
Testing Accuracy		0.19	
Training Accuracy		0.19	
Kernel		SGDClassifier	
Noise		0.5	
Training precision	0.0298	Testing Precision	0.0299
Training recall	0.126	Testing recall	0.126
Training f1 score	0.0483	Testing f1 score	0.0483

Table 33: Accuracy table for SVM with 70-30 split with noise.

Training Confusion Matrix			
Predicted class			
Actual		0	1
class	0	0	5634385
	1	1203888	173632

Table 34: Training confusion matrix for SVM with 70-30 split with noise.

Testing Confusion Matrix			
Predicted class			
Actual		0	1
class	0	0	2414659
	1	515997	74447

Table 35: Testing confusion matrix for SVM with 70-30 split with noise.

In this model the accuracy is around 19 % and precision, recall is around 3 % so, Using the SGD Classifier kernel, the SVM model was trained on a 70-30% split dataset with a high noise level. The model performs poorly on a number of evaluation measures. And its poor on unseen data.

5.2.4 SVM with 80-20 split with Noise:

The above model shows very poor accuracy with 70-30 training and testing percentage so, changed the amount of testing and training to 80-20 to see the model performance. The results are below:

SVM 80-20% split with noise			
Testing Accuracy		0.8035	
Training Accuracy		0.8033	
Kernel		SGDClassifier	
Noise		0.5	
Training precision	0	Testing Precision	0
Training recall	0	Testing recall	0
Training f1 score	0	Testing f1 score	0

Table 36: Accuracy table for SVM with 80-20 split with noise.

Training Confusion Matrix			
Predicted class			
Actual		0	1
class	0	6439370	340
	1	1573896	0

Table 37: Training confusion matrix for SVM with 80-20 split with noise.

Testing Confusion Matrix			
Predicted class			
Actual		0	1
class	0	1609254	80
	1	394068	0

Table 38: Testing confusion matrix for SVM with 80-20 split with noise.

Odd results are obtained by the SVM model trained with the SGD Classifier on an 80-20% split dataset with a 50% noise level. For both training and testing phases, the precision, recall, and F1 score metrics are all zero, despite the system attaining relatively high accuracy scores of about 80%. In both the training and testing datasets, this suggests that the model is unable to accurately identify any positive things.

Overall, this model yields very good accuracy but, however it is completely unable to detect positive incidents. A few things, like dataset imbalance, poor feature representation, or poor model selection, could be at reason for this.

5.3 Result of MLP:

5.3.1 MLP with 70-30 split without noise:

Here used 70-30 split of the data to analyse fault detection with the help of multi-layer perceptron and used optimiser as Adam as well as there are two hidden layers used for analysis MLP model. Results are shown in table no. 39.

MLP 70-30% split without noise	
Training Accuracy	0.99
Testing Accuracy	0.99
Training loss	0.0002356
Testing loss	0.000235
Hidden layer	2
Optimised	Adam

Table 39: Accuracy table for MLP with 70-30 split without noise

These results show great promise for this noise-free dataset. The model shows good accuracy in categorising cases, with both training and testing accuracies getting around 99%. Furthermore, the minimal testing and training losses (0.000235 and 0.0002356, respectively) show that the model's predictions almost match with the real target values. The presence of two hidden layers in the model architecture probably helps it recognise complicated patterns in the data. It appears that the model is effectively modifying its weights during training to reduce the loss function and enhance prediction performance because the Adam optimizer is being used.

The MLP model appears to be efficiently integrating training data and exhibiting good generalisation to previously encountered testing data, based on the overall results.

5.3.2 MLP with 80-20% split without noise:

Now changes the percentage of training and testing to see the result changes now in this model accuracies are same but, losses are different.

MLP 80-20% split without noise	
Training Accuracy	0.99
Testing Accuracy	0.99
Training loss	0.000217
Testing loss	0.0002459
Hidden layer	2
Optimised	Adam

Table 40: Accuracy table for MLP with 80-20 split without noise.

The accuracy of this model is same as 70-30 split the only difference is both losses. So, the model's ability to closely match the actual target values throughout training and testing phases is further supported by the low testing loss of 0.0002459 and training loss of 0.000217.

In conclusion, our results indicate that the MLP model with low loss values and high accuracy that was trained on this noise-free dataset performs exceptionally well in classification tasks.

5.3.3 MLP with 70-30% split with noise:

Here also checking the results with label noise in table no. 41.

MLP 70-30% split with noise	
Training Accuracy	0.7993
Testing Accuracy	0.0009
Training loss	0.5094
Testing loss	1.478
Hidden layer	2
Optimiser	Adam
Noise	0.2

Table 41: Accuracy table for MLP with 70-30 split with noise.

Good performance on the training data is shown by the relatively high training accuracy of 79.93%; however, the testing accuracy is very low at 0.09%. The significant difference implies that the model is significantly overfitting to the training set and struggles to make connections with new data. It is also evident from the testing loss of 1.478 and the training loss of 0.5094 that the model has difficulty capturing the underlying patterns in the data, especially in the testing set. The findings, considered together, indicate that the MLP model trained on this noisy dataset finds it difficult to generalise, which results in low testing accuracy and high loss values.

5.3.4 MLP with 80-20% split with noise:

Now splitting pattern is changed from 70-30 to 80-20 so we will see the change in the result. Results are as follows:

MLP 80-20% split with noise	
Training Accuracy	0.799
Testing Accuracy	0.001066
Training loss	0.5083
Testing loss	1.5898
Hidden layer	2
Optimiser	Adam
Noise	0.2

Table 42: Accuracy table for MLP with 80-20 split with noise.

Here is also training accuracy is good but testing accuracy perform very poor it means model has over fitting to training data set. According to both training and testing loss it suggests that this model underlying patterns in data. Overall, model predict moderate result.

5.3.5 MLP with 70-30% split without noise & SGD Optimizer:

MLP 70-30% split without noise	
Training Accuracy	0.99
Testing Accuracy	0.99
Training loss	0.009
Testing loss	0.0089
Hidden layer	2
Optimised	SGD

Table 43: Accuracy table for MLP with 70-30 split without noise with SGD optimizer

The accuracy is same as above model that is 99 percent. But used The Stochastic Gradient Descent (SGD) technique and the use of a two-layered hidden layer architecture are key components in the model's successful operation. More speed and better performance are made possible by the SGD optimizer, which effectively modifies the model parameters based on tiny batches of data. The MLP model trained on this noise-free dataset, according to the overall results, shows remarkable accuracy and good generalisation abilities.

5.3.6 MLP with 80-20% split without noise & SGD Optimizer:

MLP 80-20% split without noise	
Training Accuracy	0.99
Testing Accuracy	0.99
Training loss	0.00752
Testing loss	0.00747
Hidden layer	2
Optimised	SGD

Table 44: Accuracy table for MLP with 80-20 split without noise using SGD optimizer.

Both accuracies are same 99 %. Furthermore, it appears that the model's predictions in both the training and testing datasets closely match the true labels, as indicated by the low testing loss of 0.00747 and training loss of 0.00752. This implies that the model retains a high level of predictive accuracy when applied to new data and has successfully reduced errors throughout training. Overall, it is a reliable instrument for classification tasks since it efficiently discovers the underlying patterns in the data and generates extremely precise predictions.

5.3.7 MLP with 70-30% split with noise & SGD Optimizer:

MLP 70-30% split with noise	
Training Accuracy	0.7988
Testing Accuracy	0.001066
Training loss	0.00157
Testing loss	1.6409
Hidden layer	2
Optimiser	SGD
Noise	0.2

Table 45: Accuracy table for MLP with 70-30 split with noise using SGD optimizer.

Accuracy of the model is good 79 percent but, with a testing accuracy much lower than expected (around 0.11%), the model's generalisation to new data is weak. In conclusion, the MLP model that was trained using a 70-30% split with this noisy dataset shows signs of overfitting and poor generalisation to new data.

5.3.8 MLP with 80-20% split with noise & SGD Optimizer:

MLP 80-20% split with noise	
Training Accuracy	0.799
Testing Accuracy	0.00157
Training loss	0.5078
Testing loss	1.576
Hidden layer	2
Optimiser	SGD
Noise	0.2

Table 46: Accuracy table for MLP with 80-20 split with noise using SGD optimizer.

While the testing accuracy is quite low, at about 0.16%, the training accuracy is relatively good at 79.9%. This large difference in testing and training accuracy points to overfitting of the model to the training set and poor generalisation to new cases. In conclusion, an 80–20% split was used to train the MLP model on this noisy dataset. The model shows overfitting and poor generalisation to new data.

5.4 Result of DEEP MLP:

5.4.1 DEEP MLP with 70-30% split without noise:

DEEP MLP 70-30% split without noise	
Training Accuracy	0.99
Testing Accuracy	0.99
Hidden layer	5
Max_iter	1000

Table 47: Accuracy table for DEEP MLP with 70-30 split without noise.

Got 99% accuracy in both the training and testing phases is the outcome of training a deep MLP model with a 70-30% split on a noise-free dataset. The high testing accuracy shows that the model has successfully learned the patterns found in the training data and can generalise well to new instances. The model's great accuracy is attributed to the use of a deep architecture with five hidden layers, which enables it to identify complex patterns and correlations in the data.

5.4.2 DEEP MLP with 80-20% split without noise:

DEEP MLP 80-20% split without noise	
Training Accuracy	0.99
Testing Accuracy	0.99
Hidden layer	5
Max_iter	1000

Table 48:Accuracy table for DEEP MLP with 80-20 split without noise.

In this model the result is same as 70-30 split. It means both models are working same because there is no effect of train and test proportion.

5.4.3 DEEP MLP with 70-30% split with noise:

MLP 70-30% split with noise	
Training Accuracy	0.99
Testing Accuracy	0.99
Hidden layer	5
Max_iter	1000
Noise	2

Table 49: Accuracy table for DEEP MLP with 70-30 split with noise.

5.4.4 DEEP MLP with 80-20% split with noise:

MLP 80-20% split with noise	
Training Accuracy	0.99
Testing Accuracy	0.99
Hidden layer	5
Max_iter	1000
Noise	2

Table 50: Accuracy table for DEEP MLP with 80-20 split with noise.

For DEEP MLP for all split with noise and without noise the results are same the reason may be the type of the noise, the capacity of the model, and the distribution of noisy instances within the data splits are some of the possible explanations for the consistent outcomes between the two scenarios.

Based on the various models and situations discussed above, it can be observed that the MLP (Multi-Layer perceptron) with an 80-20% split without noise consistently achieves high testing and training accuracies, suggesting that it is useful for identifying faults. As an example, the MLP model with 80-20% without noise shows minimal training and testing losses (0.000217 and 0.0002459, respectively), along with testing and instruction accuracy of 0.99. This implies that the model is able to correctly identify instances that are fault-free and instances that are not, both in training and when dealing with unknown data.

5.5 Class Imbalance:

The dataset is clearly highly skewed towards the majority class (0), with 0 labels constituting 838,769 cases and 1 label comprising 209,806 instances out of a total of 1,048,575 labels. Class imbalances can be problematic for machine learning models because they can show bias towards the majority class and make it difficult for them to learn patterns from the minority class. This is especially true for models that were trained using supervised learning methods. Performance may suffer as a result, particularly on tasks like fault detection where accurately recognising instances of the minority class is crucial.

The majority class may be predicted more frequently by algorithms since class 0 has substantially greater instances than class 1. Since class 1 is a minority class with minimal representation in the dataset, algorithms may have trouble accurately predicting it. For the minority class, this may mean a decrease in precision, recall, and F1-score.

In chapter six all conclusion, recommendation and future work is discussed.

Chapter 6: Conclusion and future work

6.1 Conclusion:

One of the biggest problems facing the building management industry's stakeholders is fault identification in FCUs. This study attempted to provide a method for detecting defects in FCUs using machine learning techniques. Machine learning techniques able to identify errors in FCUs, according to an analysis of the data from the top performing machine learning models.

In conclusion, a number of Machine learning techniques were used to address the issue and identification analysis in a variety of settings, including varying data splits and noise levels. Of all the methods tested, the Multi-Layer Perceptron (MLP) proved to be the best model for fault finding time and time again. The MLP model that continuously obtained high training and testing accuracies of 0.99 with an 80-20% sample split without noise was particularly impressive. Low training and testing losses were another feature of this model that showed its resilience in identifying complex patterns in the data while preserving generalize ability to new cases. Furthermore, the MLP's accuracy in identifying fault and non-fault cases was further improved by the use of the Adam optimizer and two hidden layers. Therefore, in examination, the MLP model proves to be the best option for identifying fault.

6.2 Future Work:

In the fault detection system, there are many more things to do in the future first of all increase the data points which is main reason for getting good results, try to resolve memory issue attempting to obtain strong access that will support the application of machine learning algorithms over large amounts of data. Moreover, even better fault detection performance might result from further research into ensemble approaches. The impact of noisy data may be reduced, or the detection system's robustness may be increased by employing strategies like bagging or stacking, which aggregate predictions from several models.

Additionally, it is important to address the issue of class imbalance. By using sophisticated approaches designed to deal with imbalanced data like adaptive threshold techniques or the synthetic minority oversampling technique (SMOTE) biases may be reduced and the models' sensitivity to minority class instances increased.

References:

- [1] B. Zhang, P. Wang, G. Liu, Z. Ma, and T. Zhao, "AHU sensor fault diagnosis in various operating conditions based on a hybrid data-driven model combined energy consumption," *Journal of building engineering*, vol. 87, pp. 109028–109028, Jun. 2024, doi: <https://doi.org/10.1016/j.jobbe.2024.109028>.
- [2] Z.-C. Wang, D. Li, Z.-W. Cao, F. Gao, and M.-J. Li, "A Modified Transformer and Adapter-based Transfer Learning for Fault Detection and Diagnosis in HVAC systems," *Energy storage and saving*, Feb. 2024, doi: <https://doi.org/10.1016/j.enss.2024.02.004>.
- [3] Z. Du, B. Fan, X. Jin, and J. Chi, "Fault detection and diagnosis for buildings and HVAC systems using combined neural networks and subtractive clustering analysis," *Building and Environment*, vol. 73, pp. 1–11, Mar. 2014, doi: <https://doi.org/10.1016/j.buildenv.2013.11.021>.
- [4] J. Liang and R. Du, "Model-based Fault Detection and Diagnosis of HVAC systems using Support Vector Machine method," *International Journal of Refrigeration*, vol. 30, no. 6, pp. 1104–1114, Sep. 2007, doi: <https://doi.org/10.1016/j.ijrefrig.2006.12.012>.
- [5] Y. Zhao, C. Zhang, Y. Zhang, Z. Wang, and J. Li, "A review of data mining technologies in building energy systems: Load prediction, pattern identification, fault detection and diagnosis," *Energy and Built Environment*, vol. 1, no. 2, pp. 149–164, Apr. 2020, doi: <https://doi.org/10.1016/j.enbenv.2019.11.003>.
- [6] J. Chen, L. Zhang, Y. Li, Y. Shi, X. Gao, and Y. Hu, "A review of computing-based automated fault detection and diagnosis of heating, ventilation and air conditioning systems," *Renewable and Sustainable Energy Reviews*, vol. 161, p. 112395, Jun. 2022, doi: <https://doi.org/10.1016/j.rser.2022.112395>.
- [7] J. Bi *et al.*, "AI in HVAC fault detection and diagnosis: A systematic review," *Energy reviews*, pp. 100071–100071, Feb. 2024, doi: <https://doi.org/10.1016/j.enrev.2024.100071>.
- [8] Ahmed Faris Amiri, Houcine Oudira, Aissa Chouder, and Sofiane Kichou, "Faults detection and diagnosis of PV systems based on machine learning approach using random forest classifier," *Energy conversion and management*, vol. 301, pp. 118076–118076, Feb. 2024, doi: <https://doi.org/10.1016/j.enconman.2024.118076>.
- [9] P. Movahed, S. Taheri, and A. Razban, "A bi-level data-driven framework for fault-detection and diagnosis of HVAC systems," *Applied Energy*, vol. 339, p. 120948, Jun. 2023, doi: <https://doi.org/10.1016/j.apenergy.2023.120948>.
- [10] M. G. Albayati, J. Faraj, A. Thompson, P. Patil, R. Gorthala, and S. Rajasekaran, "Semi-Supervised Machine Learning for Fault Detection and Diagnosis of a Rooftop Unit," *Big Data Mining and Analytics*, vol. 6, no. 2, pp. 170–184, Jun. 2023, doi: <https://doi.org/10.26599/bdma.2022.9020015>.
- [11] S. Wen *et al.*, "An enhanced principal component analysis method with Savitzky–Golay filter and clustering algorithm for sensor fault detection and diagnosis," *Applied Energy*, vol. 337, pp. 120862–120862, May 2023, doi: <https://doi.org/10.1016/j.apenergy.2023.120862>.

- [12] S. Ruiz-Moreno, A. J. Gallego, A. J. Sanchez, and E. F. Camacho, "A cascade neural network methodology for fault detection and diagnosis in solar thermal plants," *Renewable Energy*, vol. 211, pp. 76–86, Jul. 2023, doi: <https://doi.org/10.1016/j.renene.2023.04.051>.
- [13] Ashraf Alghanmi and Akilu Yunusa-Kaltungo, "An ensemble-based faults detection and diagnosis approach for determining faults severities at whole-building level," *Journal of Building Performance Simulation*, vol. 17, no. 1, pp. 126–148, Aug. 2023, doi: <https://doi.org/10.1080/19401493.2023.2247382>.
- [14] K. Yan, J. Huang, Wen Zhong Shen, and Z. Ji, "Unsupervised learning for fault detection and diagnosis of air handling units," vol. 210, pp. 109689–109689, Mar. 2020, doi: <https://doi.org/10.1016/j.enbuild.2019.109689>.
- [15] R. F. R. Junior, I. A. dos S. Areias, M. M. Campos, C. E. Teixeira, L. E. B. da Silva, and G. F. Gomes, "Fault detection and diagnosis in electric motors using 1d convolutional neural networks with multi-channel vibration signals," *Measurement*, vol. 190, p. 110759, Feb. 2022, doi: <https://doi.org/10.1016/j.measurement.2022.110759>.
- [16] R. Fezai *et al.*, "Effective Random Forest-Based Fault Detection and Diagnosis for Wind Energy Conversion Systems," *IEEE Sensors Journal*, vol. 21, no. 5, pp. 6914–6921, Mar. 2021, doi: <https://doi.org/10.1109/jsen.2020.3037237>.
- [17] M. Mendoza and P. V. Tsvetkov, "An intelligent fault detection and diagnosis monitoring system for reactor operational resilience: Power transient identification," *Progress in nuclear energy*, vol. 156, pp. 104529–104529, Feb. 2023, doi: <https://doi.org/10.1016/j.pnucene.2022.104529>.
- [18] Y. Liu *et al.*, "The study of hydraulic machinery condition monitoring based on anomaly detection and fault diagnosis," *Measurement*, vol. 230, pp. 114518–114518, May 2024, doi: <https://doi.org/10.1016/j.measurement.2024.114518>.
- [19] Ashraf Alghanmi and Akilu Yunusa-Kaltungo, "A whole-building data-driven fault detection and diagnosis approach for public buildings in hot climate regions," *Energy and Built Environment*, Jul. 2023, doi: <https://doi.org/10.1016/j.enbenv.2023.07.005>.
- [20] Ahmed Faris Amiri, Houcine Oudira, Aissa Chouder, and Sofiane Kichou, "Faults detection and diagnosis of PV systems based on machine learning approach using random forest classifier," *Energy conversion and management*, vol. 301, pp. 118076–118076, Feb. 2024, doi: <https://doi.org/10.1016/j.enconman.2024.118076>.
- [21] U. Sarwar, M. Muhammad, Ainul Akmar Mokhtar, R. Khan, Paras Behrani, and Shuaib Kaka, "Hybrid intelligence for enhanced fault detection and diagnosis for industrial gas turbine engine," *Results in engineering*, vol. 21, pp. 101841–101841, Mar. 2024, doi: <https://doi.org/10.1016/j.rineng.2024.101841>.
- [22] Miloš Bjelić, Bogdan Brković, Mileta Žarković, and Tatjana Miljković, "Machine learning for power transformer SFRA based fault detection," *International journal of electrical power & energy systems*, vol. 156, pp. 109779–109779, Feb. 2024, doi: <https://doi.org/10.1016/j.ijepes.2023.109779>.

- [23] D. Bhattacharya and M. K. Nigam, "Energy efficient fault detection and classification using hyperparameter-tuned machine learning classifiers with sensors," *Measurement: Sensors*, vol. 30, p. 100908, Dec. 2023, doi: <https://doi.org/10.1016/j.measen.2023.100908>.
- [24] S. Saravanan, R. Senthil. Kumar, and P. Balakumar, "Binary firefly algorithm based reconfiguration for maximum power extraction under partial shading and machine learning approach for fault detection in solar PV arrays," *Applied soft computing*, vol. 154, pp. 111318–111318, Mar. 2024, doi: <https://doi.org/10.1016/j.asoc.2024.111318>.
- [25] A. Harish, P. Asok, and Jayan M.V, "A comparative evaluation of Stacked Auto-Encoder neural network and Multi-Layer Extreme Learning Machine for detection and classification of faults in transmission lines using WAMS data," *Energy and AI*, vol. 14, pp. 100301–100301, Oct. 2023, doi: <https://doi.org/10.1016/j.egyai.2023.100301>.
- [26] Md. Omaer. F. Goni *et al.*, "Fast and Accurate Fault Detection and Classification in Transmission Lines using Extreme Learning Machine," *e-Prime - Advances in Electrical Engineering, Electronics and Energy*, vol. 3, p. 100107, Mar. 2023, doi: <https://doi.org/10.1016/j.prime.2023.100107>.
- [27] R. Natarajan, Santosh Reddy P, Subash Chandra Bose, H.L. Gururaj, F. Flammini, and Shanmugapriya Velmurugan, "Fault detection and state estimation in robotic automatic control using machine learning," *Array*, pp. 100298–100298, Jun. 2023, doi: <https://doi.org/10.1016/j.array.2023.100298>.
- [28] Arezoo Dahesh, Reza Tavakkoli-Moghaddam, Niaz Wassan, AmirRezaTajally, Zahra Daneshi, and Aseman Erfani-Jazi, "A hybrid machine learning model based on ensemble methods for devices fault prediction in the wood industry," *Expert systems with applications*, vol. 249, pp. 123820–123820, Sep. 2024, doi: <https://doi.org/10.1016/j.eswa.2024.123820>.
- [29] Z. Soltani, K. K. Sørensen, J. Leth, and J. D. Bendtsen, "Fault detection and diagnosis in refrigeration systems using machine learning algorithms," *International Journal of Refrigeration*, vol. 144, pp. 34–45, Dec. 2022, doi: <https://doi.org/10.1016/j.ijrefrig.2022.08.008>.
- [30] B. Morris, "FAN COIL UNITS | What, Where & How - Constructandcommission.com," Jul. 11, 2021. <https://constructandcommission.com/what-is-a-fan-coil-unit/>
- [31] <https://designsolutionsmag.co.uk/fan-coil-units-detecting-fan-failure/>
- [32] U. S. A. Coil, "Common Problems with Fan Coil Units in Residences," *USA Coil & Air*, Jan. 02, 2019. <https://usacoil.com/common-problems-with-fan-coil-units/>
- [33] Q. Nguyen, Tomoaki Shikina, D. Teruya, S. Hotta, H.-D. Han, and Hironori Nakajo, "Leveraging Expert Knowledge for Label Noise Mitigation in Machine Learning," *Applied sciences*, vol. 11, no. 22, pp. 11040–11040, Nov. 2021, doi: <https://doi.org/10.3390/app112211040>.
- [34] "Figure 2. Classification of data by support vector machine (SVM).," *ResearchGate*. https://www.researchgate.net/figure/Classification-of-data-by-support-vector-machine-SVM_fig8_304611323

Appendix:

1. Value column into feature vector transformation

```
import pandas as pd
import numpy as np

# Load the dataset
data = pd.read_csv('C:\\Users\\Owner\\Downloads\\100
files\\merged_100_files.csv')

# Define the bins based on the actual values in the 'value' column
bins = np.linspace(data['value'].min(), data['value'].max(), 11)

# Create the 10 feature vectors based on the 'value' column
for i in range(1, 11):
    bin_name = f'value_bin_{i}'
    # Assign the actual values to the corresponding bin column
    data[bin_name] = np.where((data['value'] >= bins[i-1]) &
(data['value'] < bins[i]), data['value'], np.nan)

# Fill NaN values with the actual values from the 'value' column
for i in range(1, 11):
    bin_name = f'value_bin_{i}'
    data[bin_name] = data[bin_name].fillna(data['value'])

# Save the modified DataFrame to a new CSV file
data.to_csv('C:\\Users\\Owner\\Downloads\\100
files\\merged_100_files_with_bins.csv', index=False)

# Display the first few rows to verify the transformation
print(data.head())
```

2. KNN with 70-30 split without noise:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
# Load the data
data=pd.read_csv('C:\\Users\\Owner\\Downloads\\100files\\100filesyour_modified_data_with_labels.csv')
# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']
# Encode categorical features if any
label_encoder = LabelEncoder()
for col in X.select_dtypes(include='object').columns:
    X[col] = label_encoder.fit_transform(X[col])
# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Initialize KNN classifier
knn_classifier = KNeighborsClassifier()
# Train the classifier
knn_classifier.fit(X_train_scaled, y_train)
# Predict on training set
y_train_pred = knn_classifier.predict(X_train_scaled)
# Predict on test set
y_test_pred = knn_classifier.predict(X_test_scaled)
# Calculate accuracy
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)
print("Training Accuracy:", train_accuracy)
print("Testing Accuracy:", test_accuracy)

```

3. KNN with 80-20 split without noise:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
# Load the data
data = pd.read_csv('C:\\Users\\Owner\\Downloads\\100 files\\100 filesyour_modified_data_with_labels.csv')
# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']
# Encode categorical features if any
label_encoder = LabelEncoder()
for col in X.select_dtypes(include='object').columns:
    X[col] = label_encoder.fit_transform(X[col])
# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

```

# Initialize KNN classifier
knn_classifier = KNeighborsClassifier()
# Train the classifier
knn_classifier.fit(X_train_scaled, y_train)
# Predict on training set
y_train_pred = knn_classifier.predict(X_train_scaled)
# Predict on test set
y_test_pred = knn_classifier.predict(X_test_scaled)
# Calculate accuracy
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)
print("Training Accuracy:", train_accuracy)
print("Testing Accuracy:", test_accuracy)

```

4. KNN with 70-30 split WITH noise:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load the data
data = pd.read_csv('C:\\Users\\Owner\\Downloads\\100 files\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']
# Encode categorical features if any
label_encoder = LabelEncoder()
for col in X.select_dtypes(include='object').columns:
    X[col] = label_encoder.fit_transform(X[col])

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Define the percentage of labels to flip
noise_level = 0.5
# Randomly select indices to flip labels
import numpy as np
num_samples_to_flip = int(noise_level * len(y_train))
indices_to_flip = np.random.choice(len(y_train), num_samples_to_flip,
replace=False)
# Flip the labels at selected indices
y_train_noisy = y_train.copy()
y_train_noisy.iloc[indices_to_flip] = np.random.randint(2,
size=num_samples_to_flip) # Randomly flip labels
# Initialize KNN classifier
knn_classifier = KNeighborsClassifier()
# Train the classifier with noisy labels
knn_classifier.fit(X_train_scaled, y_train_noisy)
# Predict on training set
y_train_pred = knn_classifier.predict(X_train_scaled)

```

```
# Predict on test set
y_test_pred = knn_classifier.predict(X_test_scaled)
# Calculate accuracy
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)
print("Training Accuracy:", train_accuracy)
print("Testing Accuracy:", test_accuracy)
```

5. KNN with 80-20 split with noise:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load the data
data = pd.read_csv('C:\\Users\\Owner\\Downloads\\100 files\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']
# Encode categorical features if any
label_encoder = LabelEncoder()
for col in X.select_dtypes(include='object').columns:
    X[col] = label_encoder.fit_transform(X[col])

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Define the percentage of labels to flip
noise_level = 0.5
# Randomly select indices to flip labels
import numpy as np
num_samples_to_flip = int(noise_level * len(y_train))
indices_to_flip = np.random.choice(len(y_train), num_samples_to_flip,
replace=False)
# Flip the labels at selected indices
y_train_noisy = y_train.copy()
y_train_noisy.iloc[indices_to_flip] = np.random.randint(2,
size=num_samples_to_flip) # Randomly flip labels
# Initialize KNN classifier
knn_classifier = KNeighborsClassifier()
# Train the classifier with noisy labels
knn_classifier.fit(X_train_scaled, y_train_noisy)
# Predict on training set
y_train_pred = knn_classifier.predict(X_train_scaled)
# Predict on test set
y_test_pred = knn_classifier.predict(X_test_scaled)
# Calculate accuracy
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)
print("Training Accuracy:", train_accuracy)
```

```
print("Testing Accuracy:", test_accuracy)
```

6. KNN with 70-30 split without noise (N=10)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load the data
data = pd.read_csv('C:\\Users\\student\\Downloads\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Encode categorical features if any
label_encoder = LabelEncoder()
for col in X.select_dtypes(include='object').columns:
    X[col] = label_encoder.fit_transform(X[col])

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Initialize KNN classifier with k=10
knn_classifier = KNeighborsClassifier(n_neighbors=10)

# Train the classifier
knn_classifier.fit(X_train_scaled, y_train)

# Predict on training set
y_train_pred = knn_classifier.predict(X_train_scaled)

# Predict on test set
y_test_pred = knn_classifier.predict(X_test_scaled)

# Calculate accuracy
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

print("Training Accuracy:", train_accuracy)
print("Testing Accuracy:", test_accuracy)
```

7. KNN with 80-20 split without noise (N=10)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load the data
data = pd.read_csv('C:\\Users\\student\\Downloads\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Encode categorical features if any
label_encoder = LabelEncoder()
for col in X.select_dtypes(include='object').columns:
    X[col] = label_encoder.fit_transform(X[col])

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Initialize KNN classifier with k=10
knn_classifier = KNeighborsClassifier(n_neighbors=10)

# Train the classifier
knn_classifier.fit(X_train_scaled, y_train)

# Predict on training set
y_train_pred = knn_classifier.predict(X_train_scaled)

# Predict on test set
y_test_pred = knn_classifier.predict(X_test_scaled)

# Calculate accuracy
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

print("Training Accuracy:", train_accuracy)
print("Testing Accuracy:", test_accuracy)

```

8. KNN with 70-30 split with noise (N=10)

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
import numpy as np

# Load the data
data = pd.read_csv('C:\\Users\\student\\Downloads\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)

```

```

y = data['label']

# Encode categorical features if any
label_encoder = LabelEncoder()
for col in X.select_dtypes(include='object').columns:
    X[col] = label_encoder.fit_transform(X[col])

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Initialize KNN classifier with k=10
knn_classifier = KNeighborsClassifier(n_neighbors=10)

# Define the percentage of labels to flip
noise_level = 0.5

# Randomly select indices to flip labels
num_samples_to_flip = int(noise_level * len(y_train))
indices_to_flip = np.random.choice(len(y_train), num_samples_to_flip,
replace=False)

# Flip the labels at selected indices
y_train_noisy = y_train.copy()
y_train_noisy.iloc[indices_to_flip] = np.random.randint(2,
size=num_samples_to_flip)

# Train the classifier with noisy labels
knn_classifier.fit(X_train_scaled, y_train_noisy)

# Predict on training set
y_train_pred = knn_classifier.predict(X_train_scaled)

# Predict on test set
y_test_pred = knn_classifier.predict(X_test_scaled)

# Calculate accuracy
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

print("Training Accuracy:", train_accuracy)
print("Testing Accuracy:", test_accuracy)

```

9. KNN with 80-20 split with noise (N=10)

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
import numpy as np

# Load the data

```

```

data = pd.read_csv('C:\\Users\\student\\Downloads\\100
filesyour_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Encode categorical features if any
label_encoder = LabelEncoder()
for col in X.select_dtypes(include='object').columns:
    X[col] = label_encoder.fit_transform(X[col])

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Initialize KNN classifier with k=10
knn_classifier = KNeighborsClassifier(n_neighbors=10)

# Define the percentage of labels to flip
noise_level = 0.5

# Randomly select indices to flip labels
num_samples_to_flip = int(noise_level * len(y_train))
indices_to_flip = np.random.choice(len(y_train), num_samples_to_flip,
replace=False)

# Flip the labels at selected indices
y_train_noisy = y_train.copy()
y_train_noisy.iloc[indices_to_flip] = np.random.randint(2,
size=num_samples_to_flip)

# Train the classifier with noisy labels
knn_classifier.fit(X_train_scaled, y_train_noisy)

# Predict on training set
y_train_pred = knn_classifier.predict(X_train_scaled)

# Predict on test set
y_test_pred = knn_classifier.predict(X_test_scaled)

# Calculate accuracy
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

print("Training Accuracy:", train_accuracy)
print("Testing Accuracy:", test_accuracy)

```

10. SVM with 70-30 without noise:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

```

```

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
# Load the data
data = pd.read_csv('C:\\Users\\Owner\\Downloads\\100 files\\100
files\\your_modified_data_with_labels.csv')
# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
# Define columns for different types of preprocessing
numeric_features = ['value', 'value_bin_1', 'value_bin_2', 'value_bin_3',
'value_bin_4',
'value_bin_5', 'value_bin_6', 'value_bin_7', 'value_bin_8',
'value_bin_9', 'value_bin_10']
# Define preprocessing steps for numeric features
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features)])
# Apply preprocessing to the data
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)
# Initialize SVM classifier
svm_classifier = SVC(kernel='linear', random_state=42)
# Train the classifier
svm_classifier.fit(X_train_processed, y_train)
# Predict on training set
y_train_pred = svm_classifier.predict(X_train_processed)
train_accuracy = accuracy_score(y_train, y_train_pred)

# Predict on test set
y_test_pred = svm_classifier.predict(X_test_processed)
test_accuracy = accuracy_score(y_test, y_test_pred)
print("Training Accuracy:", train_accuracy)
print("Testing Accuracy:", test_accuracy)

```

11. SVM with 80-20 split without noise:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
# Load the data
data = pd.read_csv('C:\\Users\\Owner\\Downloads\\100 files\\100
files\\your_modified_data_with_labels.csv')
# Define features and target

```

```

X = data.drop(['label'], axis=1)
y = data['label']

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
# Define columns for different types of preprocessing
numeric_features = ['value', 'value_bin_1', 'value_bin_2', 'value_bin_3',
'value_bin_4',
'value_bin_5', 'value_bin_6', 'value_bin_7', 'value_bin_8',
'value_bin_9', 'value_bin_10']
# Define preprocessing steps for numeric features
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features)])
# Apply preprocessing to the data
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)
# Initialize SVM classifier
svm_classifier = SVC(kernel='linear', random_state=42)
# Train the classifier
svm_classifier.fit(X_train_processed, y_train)
# Predict on training set
y_train_pred = svm_classifier.predict(X_train_processed)
train_accuracy = accuracy_score(y_train, y_train_pred)

# Predict on test set
y_test_pred = svm_classifier.predict(X_test_processed)
test_accuracy = accuracy_score(y_test, y_test_pred)
print("Training Accuracy:", train_accuracy)
print("Testing Accuracy:", test_accuracy)

```

12. SVM with 70-30 split with noise:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
import numpy as np

# Load the data
data = pd.read_csv('C:\\Users\\student\\Downloads\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Split the data into train and test sets

```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Define columns for different types of preprocessing
numeric_features = ['value', 'value_bin_1', 'value_bin_2', 'value_bin_3',
'value_bin_4',
'value_bin_5', 'value_bin_6', 'value_bin_7', 'value_bin_8',
'value_bin_9', 'value_bin_10']

# Define preprocessing steps for numeric features
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features)])

# Apply preprocessing to the data
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

# Define the percentage of labels to flip
noise_level = 0.5# You can adjust this value based on the amount of noise
you want to introduce

# Randomly select indices to flip labels
num_samples_to_flip = int(noise_level * len(y_train))
indices_to_flip = np.random.choice(len(y_train), num_samples_to_flip,
replace=False)

# Flip the labels at selected indices
y_train_noisy = y_train.copy()
y_train_noisy.iloc[indices_to_flip] = 1 -
y_train_noisy.iloc[indices_to_flip] # Flip binary labels

fromsklearn.linear_modelimportSGDClassifier

# Initialize SVM classifier with SGD
svm_classifier = SGDClassifier(loss='hinge', random_state=42)

# Train the classifier with noisy labels
svm_classifier.fit(X_train_processed, y_train_noisy)

# Predict on training set
y_train_pred = svm_classifier.predict(X_train_processed)
train_accuracy = accuracy_score(y_train, y_train_pred)

# Predict on test set
y_test_pred = svm_classifier.predict(X_test_processed)
test_accuracy = accuracy_score(y_test, y_test_pred)

print("Training Accuracy:", train_accuracy)
print("Testing Accuracy:", test_accuracy)

```

13. SVM with 80-20 split with noise:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
import numpy as np

# Load the data
data = pd.read_csv('C:\\Users\\student\\Downloads\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Define columns for different types of preprocessing
numeric_features = ['value', 'value_bin_1', 'value_bin_2', 'value_bin_3',
'value_bin_4',
'value_bin_5', 'value_bin_6', 'value_bin_7', 'value_bin_8',
'value_bin_9', 'value_bin_10']

# Define preprocessing steps for numeric features
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features)])

# Apply preprocessing to the data
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

# Define the percentage of labels to flip
noise_level = 0.5 # You can adjust this value based on the amount of noise
you want to introduce

# Randomly select indices to flip labels
num_samples_to_flip = int(noise_level * len(y_train))
indices_to_flip = np.random.choice(len(y_train), num_samples_to_flip,
replace=False)

# Flip the labels at selected indices
y_train_noisy = y_train.copy()
y_train_noisy.iloc[indices_to_flip] = 1 -
y_train_noisy.iloc[indices_to_flip] # Flip binary labels

from sklearn.linear_model import SGDClassifier

# Initialize SVM classifier with SGD
```

```

svm_classifier = SGDClassifier(loss='hinge', random_state=42)

# Train the classifier with noisy labels
svm_classifier.fit(X_train_processed, y_train_noisy)

# Predict on training set
y_train_pred = svm_classifier.predict(X_train_processed)
train_accuracy = accuracy_score(y_train, y_train_pred)

# Predict on test set
y_test_pred = svm_classifier.predict(X_test_processed)
test_accuracy = accuracy_score(y_test, y_test_pred)

print("Training Accuracy:", train_accuracy)
print("Testing Accuracy:", test_accuracy)

```

14. MLP with 70-30 split with noise (optimizer= adam)

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras import regularizers
import numpy as np

# Load the data
data = pd.read_csv('C:\\Users\\Owner\\Downloads\\100 files\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Define columns for different types of preprocessing
numeric_features = ['value', 'value_bin_1', 'value_bin_2', 'value_bin_3',
'value_bin_4',
'value_bin_5', 'value_bin_6', 'value_bin_7', 'value_bin_8',
'value_bin_9', 'value_bin_10']

# Define preprocessing steps for numeric features
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features)])

# Apply preprocessing to the data
X_train_processed = preprocessor.fit_transform(X_train)

```

```

X_test_processed = preprocessor.transform(X_test)

# Introduce label noise
noise_level = 0.2
y_train_noisy = np.random.binomial(n=1, p=1-noise_level,
size=y_train.shape) # Flip labels with probability noise_level
y_train_noisy = np.logical_xor(y_train_noisy, y_train) # Introduce label
noise

# Build the MLP model with L2 regularization
model = Sequential([
Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.01),
input_shape=(X_train_processed.shape[1],)),
Dropout(0.5), # Dropout layer for regularization
Dense(32, activation='relu', kernel_regularizer=regularizers.l2(0.01)),
Dropout(0.5),
Dense(1, activation='sigmoid')
])

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

# Train the model
history = model.fit(X_train_processed, y_train_noisy, epochs=5,
validation_data=(X_test_processed, y_test))

# Evaluate the model
train_loss, train_accuracy = model.evaluate(X_train_processed,
y_train_noisy)
test_loss, test_accuracy = model.evaluate(X_test_processed, y_test)

print("Training Loss:", train_loss)
print("Training Accuracy:", train_accuracy)
print("Testing Loss:", test_loss)
print("Testing Accuracy:", test_accuracy)

```

15. MLP with 80-20 split with noise (optimizer = adam):

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras import regularizers
import numpy as np

# Load the data
data = pd.read_csv('C:\\Users\\student\\Downloads\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Split the data into train and test sets

```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Define columns for different types of preprocessing
numeric_features = ['value', 'value_bin_1', 'value_bin_2', 'value_bin_3',
'value_bin_4',
'value_bin_5', 'value_bin_6', 'value_bin_7', 'value_bin_8',
'value_bin_9', 'value_bin_10']

# Define preprocessing steps for numeric features
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features)])

# Apply preprocessing to the data
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

# Introduce label noise
noise_level = 0.2
y_train_noisy = np.random.binomial(n=1, p=1-noise_level,
size=y_train.shape) # Flip labels with probability noise_level
y_train_noisy = np.logical_xor(y_train_noisy, y_train) # Introduce label
noise

# Build the MLP model with L2 regularization
model = Sequential([
Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.01),
input_shape=(X_train_processed.shape[1],)),
Dropout(0.5), # Dropout layer for regularization
Dense(32, activation='relu', kernel_regularizer=regularizers.l2(0.01)),
Dropout(0.5),
Dense(1, activation='sigmoid')
])

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

# Train the model
history = model.fit(X_train_processed, y_train_noisy, epochs=10,
validation_data=(X_test_processed, y_test))

# Evaluate the model
train_loss, train_accuracy = model.evaluate(X_train_processed,
y_train_noisy)
test_loss, test_accuracy = model.evaluate(X_test_processed, y_test)

print("Training Loss:", train_loss)
print("Training Accuracy:", train_accuracy)
print("Testing Loss:", test_loss)
print("Testing Accuracy:", test_accuracy)

```

16. MLP with 70-30 without noise

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Load the data
data = pd.read_csv('C:\\Users\\Owner\\Downloads\\100 files\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Define columns for different types of preprocessing
numeric_features = ['value', 'value_bin_1', 'value_bin_2', 'value_bin_3',
'value_bin_4',
'value_bin_5', 'value_bin_6', 'value_bin_7', 'value_bin_8',
'value_bin_9', 'value_bin_10']

# Define preprocessing steps for numeric features
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features)])

# Apply preprocessing to the data
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

# Build the MLP model
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train_processed.shape[1],)),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

# Train the model
history = model.fit(X_train_processed, y_train, epochs=5,
validation_data=(X_test_processed, y_test))

# Evaluate the model
train_loss, train_accuracy = model.evaluate(X_train_processed, y_train)
test_loss, test_accuracy = model.evaluate(X_test_processed, y_test)
```

```

print("Training Loss:", train_loss)
print("Training Accuracy:", train_accuracy)
print("Testing Loss:", test_loss)
print("Testing Accuracy:", test_accuracy)

```

17. MLP with 80-20 without noise:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Load the data
data = pd.read_csv('C:\\Users\\Owner\\Downloads\\100 files\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Define columns for different types of preprocessing
numeric_features = ['value', 'value_bin_1', 'value_bin_2', 'value_bin_3',
'value_bin_4',
'value_bin_5', 'value_bin_6', 'value_bin_7', 'value_bin_8',
'value_bin_9', 'value_bin_10']

# Define preprocessing steps for numeric features
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features)])

# Apply preprocessing to the data
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

# Build the MLP model
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train_processed.shape[1],)),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

```

```

# Train the model
history = model.fit(X_train_processed, y_train, epochs=5,
validation_data=(X_test_processed, y_test))

# Evaluate the model
train_loss, train_accuracy = model.evaluate(X_train_processed, y_train)
test_loss, test_accuracy = model.evaluate(X_test_processed, y_test)

print("Training Loss:", train_loss)
print("Training Accuracy:", train_accuracy)
print("Testing Loss:", test_loss)
print("Testing Accuracy:", test_accuracy)

```

18. MLP with 70-30 split with SGD optimizer:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras import regularizers
import numpy as np
from tensorflow.keras.optimizers import SGD

# Load the data
data = pd.read_csv('C:\\Users\\student\\Downloads\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Define columns for different types of preprocessing
numeric_features = ['value', 'value_bin_1', 'value_bin_2', 'value_bin_3',
'value_bin_4',
'value_bin_5', 'value_bin_6', 'value_bin_7', 'value_bin_8',
'value_bin_9', 'value_bin_10']

# Define preprocessing steps for numeric features
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features)])

# Apply preprocessing to the data
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

```

```

# Build the MLP model with SGD optimizer
model_sgd = Sequential([
Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.01),
input_shape=(X_train_processed.shape[1],)),
Dropout(0.5), # Dropout layer for regularization
Dense(32, activation='relu', kernel_regularizer=regularizers.l2(0.01)),
Dropout(0.5),
Dense(1, activation='sigmoid')
])

# Compile the model with SGD optimizer
model_sgd.compile(optimizer=SGD(learning_rate=0.01, momentum=0.9),
loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
history_sgd = model_sgd.fit(X_train_processed, y_train, epochs=5,
validation_data=(X_test_processed, y_test))

# Evaluate the model
train_loss_sgd, train_accuracy_sgd =
model_sgd.evaluate(X_train_processed, y_train)
test_loss_sgd, test_accuracy_sgd = model_sgd.evaluate(X_test_processed,
y_test)

print("SGD Optimizer Results (Without Noise):")
print("Training Loss:", train_loss_sgd)
print("Training Accuracy:", train_accuracy_sgd)
print("Testing Loss:", test_loss_sgd)
print("Testing Accuracy:", test_accuracy_sgd)

```

19. MLP with 80-20 split with SGD optimizer without noise:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras import regularizers
import numpy as np
from tensorflow.keras.optimizers import SGD

# Load the data
data = pd.read_csv('C:\\Users\\student\\Downloads\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Define columns for different types of preprocessing

```

```

numeric_features = ['value', 'value_bin_1', 'value_bin_2', 'value_bin_3',
'value_bin_4',
'value_bin_5', 'value_bin_6', 'value_bin_7', 'value_bin_8',
'value_bin_9', 'value_bin_10']

# Define preprocessing steps for numeric features
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features)])

# Apply preprocessing to the data
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

# Build the MLP model with SGD optimizer
model_sgd = Sequential([
Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.01),
input_shape=(X_train_processed.shape[1],)),
Dropout(0.5), # Dropout layer for regularization
Dense(32, activation='relu', kernel_regularizer=regularizers.l2(0.01)),
Dropout(0.5),
Dense(1, activation='sigmoid')
])

# Compile the model with SGD optimizer
model_sgd.compile(optimizer=SGD(learning_rate=0.01, momentum=0.9),
loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
history_sgd = model_sgd.fit(X_train_processed, y_train, epochs=5,
validation_data=(X_test_processed, y_test))

# Evaluate the model
train_loss_sgd, train_accuracy_sgd =
model_sgd.evaluate(X_train_processed, y_train)
test_loss_sgd, test_accuracy_sgd = model_sgd.evaluate(X_test_processed,
y_test)

print("SGD Optimizer Results (Without Noise):")
print("Training Loss:", train_loss_sgd)
print("Training Accuracy:", train_accuracy_sgd)
print("Testing Loss:", test_loss_sgd)
print("Testing Accuracy:", test_accuracy_sgd)

```

20. MLP with 70-30 split with noise & SGD optimizer

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

```

```

from tensorflow.keras import regularizers
import numpy as np
from tensorflow.keras.optimizers import SGD

# Load the data
data = pd.read_csv('C:\\Users\\student\\Downloads\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Define columns for different types of preprocessing
numeric_features = ['value', 'value_bin_1', 'value_bin_2', 'value_bin_3',
'value_bin_4',
'value_bin_5', 'value_bin_6', 'value_bin_7', 'value_bin_8',
'value_bin_9', 'value_bin_10']

# Define preprocessing steps for numeric features
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features)])

# Apply preprocessing to the data
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

# Introduce label noise
noise_level = 0.2
y_train_noisy = np.random.binomial(n=1, p=1-noise_level,
size=y_train.shape) # Flip labels with probability noise_level
y_train_noisy = np.logical_xor(y_train_noisy, y_train) # Introduce label
noise

# Build the MLP model with SGD optimizer
model_sgd = Sequential([
    Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.01),
input_shape=(X_train_processed.shape[1],)),
    Dropout(0.5), # Dropout layer for regularization
    Dense(32, activation='relu', kernel_regularizer=regularizers.l2(0.01)),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

# Compile the model with SGD optimizer
model_sgd.compile(optimizer=SGD(learning_rate=0.01, momentum=0.9),
loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
history_sgd = model_sgd.fit(X_train_processed, y_train_noisy, epochs=5,
validation_data=(X_test_processed, y_test))

```

```
# Evaluate the model
train_loss_sgd, train_accuracy_sgd =
model_sgd.evaluate(X_train_processed, y_train_noisy)
test_loss_sgd, test_accuracy_sgd = model_sgd.evaluate(X_test_processed,
y_test)

print("SGD Optimizer Results:")
print("Training Loss:", train_loss_sgd)
print("Training Accuracy:", train_accuracy_sgd)
print("Testing Loss:", test_loss_sgd)
print("Testing Accuracy:", test_accuracy_sgd)
```

21. MLP with 80-20 split with noise & SGD optimizer

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras import regularizers
import numpy as np
from tensorflow.keras.optimizers import SGD

# Load the data
data = pd.read_csv('C:\\Users\\student\\Downloads\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Define columns for different types of preprocessing
numeric_features = ['value', 'value_bin_1', 'value_bin_2', 'value_bin_3',
'value_bin_4',
'value_bin_5', 'value_bin_6', 'value_bin_7', 'value_bin_8',
'value_bin_9', 'value_bin_10']

# Define preprocessing steps for numeric features
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features)])

# Apply preprocessing to the data
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

# Introduce label noise
```

```

noise_level = 0.2
y_train_noisy = np.random.binomial(n=1, p=1-noise_level,
size=y_train.shape) # Flip labels with probability noise_level
y_train_noisy = np.logical_xor(y_train_noisy, y_train) # Introduce label
noise

# Build the MLP model with SGD optimizer
model_sgd = Sequential([
Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.01),
input_shape=(X_train_processed.shape[1],)),
Dropout(0.5), # Dropout layer for regularization
Dense(32, activation='relu', kernel_regularizer=regularizers.l2(0.01)),
Dropout(0.5),
Dense(1, activation='sigmoid')
])

# Compile the model with SGD optimizer
model_sgd.compile(optimizer=SGD(learning_rate=0.01, momentum=0.9),
loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
history_sgd = model_sgd.fit(X_train_processed, y_train_noisy, epochs=5,
validation_data=(X_test_processed, y_test))

# Evaluate the model
train_loss_sgd, train_accuracy_sgd =
model_sgd.evaluate(X_train_processed, y_train_noisy)
test_loss_sgd, test_accuracy_sgd = model_sgd.evaluate(X_test_processed,
y_test)

print("SGD Optimizer Results:")
print("Training Loss:", train_loss_sgd)
print("Training Accuracy:", train_accuracy_sgd)
print("Testing Loss:", test_loss_sgd)
print("Testing Accuracy:", test_accuracy_sgd)

```

22. DEEP MLP with 70-30 split without noise

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
import numpy as np

# Load the data
data = pd.read_csv('C:\\Users\\student\\Downloads\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

```

```

# Define columns for different types of preprocessing
numeric_features = ['value', 'value_bin_1', 'value_bin_2', 'value_bin_3',
'value_bin_4',
'value_bin_5', 'value_bin_6', 'value_bin_7', 'value_bin_8',
'value_bin_9', 'value_bin_10']

# Preprocess numeric columns
numeric_transformer = Pipeline(steps=[
    ('scaler', StandardScaler())
])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features)
    ])

# Apply preprocessing to the data
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

# Define the percentage of labels to flip
noise_level = 0.2

# Randomly select indices to flip labels
num_samples_to_flip = int(noise_level * len(y_train))
indices_to_flip = np.random.choice(len(y_train), num_samples_to_flip,
replace=False)

# Flip the labels at selected indices
y_train_noisy = y_train.copy().to_numpy()
y_train_noisy[indices_to_flip] = 1 - y_train_noisy[indices_to_flip] #
Flip binary labels

# Initialize MLP classifier with 5 hidden layers
mlp_classifier = MLPClassifier(hidden_layer_sizes=(128, 64, 32, 16, 8),
max_iter=1000, random_state=42)

# Train the classifier
mlp_classifier.fit(X_train_processed, y_train_noisy)

# Predict on training set
y_train_pred = mlp_classifier.predict(X_train_processed)

# Evaluate train accuracy
train_accuracy = accuracy_score(y_train, y_train_pred)

# Predict on testing set
y_test_pred = mlp_classifier.predict(X_test_processed)

# Evaluate test accuracy
test_accuracy = accuracy_score(y_test, y_test_pred)

print("Training Accuracy with Noise 0.2:", train_accuracy)
print("Testing Accuracy with Noise 0.2:", test_accuracy)

```

23. DEEP MLP with 80-20 split without noise

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
import numpy as np

# Load the data
data = pd.read_csv('C:\\Users\\student\\Downloads\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Define columns for different types of preprocessing
numeric_features = ['value', 'value_bin_1', 'value_bin_2', 'value_bin_3',
'value_bin_4',
'value_bin_5', 'value_bin_6', 'value_bin_7', 'value_bin_8',
'value_bin_9', 'value_bin_10']

# Preprocess numeric columns
numeric_transformer = Pipeline(steps=[
    ('scaler', StandardScaler())
])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features)
    ])

# Apply preprocessing to the data
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

# Define the percentage of labels to flip
noise_level = 0.2

# Randomly select indices to flip labels
num_samples_to_flip = int(noise_level * len(y_train))
indices_to_flip = np.random.choice(len(y_train), num_samples_to_flip,
replace=False)

# Flip the labels at selected indices
y_train_noisy = y_train.copy().to_numpy()
y_train_noisy[indices_to_flip] = 1 - y_train_noisy[indices_to_flip] #
Flip binary labels

# Initialize MLP classifier with 5 hidden layers
mlp_classifier = MLPClassifier(hidden_layer_sizes=(128, 64, 32, 16, 8),
max_iter=1000, random_state=42)
```

```

# Train the classifier
mlp_classifier.fit(X_train_processed, y_train_noisy)

# Predict on training set
y_train_pred = mlp_classifier.predict(X_train_processed)

# Evaluate train accuracy
train_accuracy = accuracy_score(y_train, y_train_pred)

# Predict on testing set
y_test_pred = mlp_classifier.predict(X_test_processed)

# Evaluate test accuracy
test_accuracy = accuracy_score(y_test, y_test_pred)

print("Training Accuracy with Noise 0.2:", train_accuracy)
print("Testing Accuracy with Noise 0.2:", test_accuracy)

```

24. DEEP MLP 70-30 split with noise:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
import numpy as np

# Load the data
data = pd.read_csv('C:\\Users\\Owner\\Downloads\\100 files\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Define columns for different types of preprocessing
numeric_features = ['value', 'value_bin_1', 'value_bin_2', 'value_bin_3',
'value_bin_4',
'value_bin_5', 'value_bin_6', 'value_bin_7', 'value_bin_8',
'value_bin_9', 'value_bin_10']

# Preprocess numeric columns
numeric_transformer = Pipeline(steps=[
    ('scaler', StandardScaler())
])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features)
    ])

# Apply preprocessing to the data

```

```

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

# Define the percentage of labels to flip
noise_level = 0.2

# Randomly select indices to flip labels
num_samples_to_flip = int(noise_level * len(y_train))
indices_to_flip = np.random.choice(len(y_train), num_samples_to_flip,
replace=False)

# Flip the labels at selected indices
y_train_noisy = y_train.copy().to_numpy()
y_train_noisy[indices_to_flip] = 1 - y_train_noisy[indices_to_flip] #
Flip binary labels

# Initialize MLP classifier with 5 hidden layers
mlp_classifier = MLPClassifier(hidden_layer_sizes=(128, 64, 32, 16, 8),
max_iter=1000, random_state=42)

# Train the classifier
mlp_classifier.fit(X_train_processed, y_train_noisy)

# Predict on training set
y_train_pred = mlp_classifier.predict(X_train_processed)

# Evaluate train accuracy
train_accuracy = accuracy_score(y_train, y_train_pred)

# Predict on testing set
y_test_pred = mlp_classifier.predict(X_test_processed)

# Evaluate test accuracy
test_accuracy = accuracy_score(y_test, y_test_pred)

print("Training Accuracy with Noise 0.2:", train_accuracy)
print("Testing Accuracy with Noise 0.2:", test_accuracy)

```

25. DEEP MLP with 80-20 split with noise

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
import numpy as np

# Load the data
data = pd.read_csv('C:\\Users\\Owner\\Downloads\\100 files\\100
files\\your_modified_data_with_labels.csv')

# Define features and target
X = data.drop(['label'], axis=1)
y = data['label']

# Split the data into train and test sets

```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Define columns for different types of preprocessing
numeric_features = ['value', 'value_bin_1', 'value_bin_2', 'value_bin_3',
'value_bin_4',
'value_bin_5', 'value_bin_6', 'value_bin_7', 'value_bin_8',
'value_bin_9', 'value_bin_10']

# Preprocess numeric columns
numeric_transformer = Pipeline(steps=[
    ('scaler', StandardScaler())
])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features)
    ])

# Apply preprocessing to the data
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

# Define the percentage of labels to flip
noise_level = 0.2

# Randomly select indices to flip labels
num_samples_to_flip = int(noise_level * len(y_train))
indices_to_flip = np.random.choice(len(y_train), num_samples_to_flip,
replace=False)

# Flip the labels at selected indices
y_train_noisy = y_train.copy().to_numpy()
y_train_noisy[indices_to_flip] = 1 - y_train_noisy[indices_to_flip] #
Flip binary labels

# Initialize MLP classifier with 5 hidden layers
mlp_classifier = MLPClassifier(hidden_layer_sizes=(128, 64, 32, 16, 8),
max_iter=1000, random_state=42)

# Train the classifier
mlp_classifier.fit(X_train_processed, y_train_noisy)

# Predict on training set
y_train_pred = mlp_classifier.predict(X_train_processed)

# Evaluate train accuracy
train_accuracy = accuracy_score(y_train, y_train_pred)

# Predict on testing set
y_test_pred = mlp_classifier.predict(X_test_processed)

# Evaluate test accuracy
test_accuracy = accuracy_score(y_test, y_test_pred)

print("Training Accuracy with Noise 0.2:", train_accuracy)
print("Testing Accuracy with Noise 0.2:", test_accuracy)

```